

Arrays in Python

Technical Notes

Rodrigo Kang

Table of contents

1	Arrays	4
1.1	NumPy and the ndarray	4
1.2	Creating Arrays	6
1.2.1	Creating Arrays from Python Sequences	6
1.2.2	Homogeneous Data Types	7
1.3	Dimensions	8
1.3.1	Zero-Dimensional Arrays	8
1.3.2	One-Dimensional Arrays	9
1.3.3	Two-Dimensional Arrays	9
1.3.4	Higher-Dimensional Arrays	10
1.4	Shape	10
1.4.1	Shape as Structural Information	12
1.5	Size	13
1.6	Data Type	14
1.7	Creating Structured Arrays	15
1.7.1	Arrays of Zeros	16
1.7.2	Arrays of Ones	16
1.7.3	Arrays Filled with a Constant	17
1.7.4	Numerical Sequences with arange()	17
1.7.5	Evenly Spaced Values with linspace()	18
1.7.6	Identity Matrices	19
1.7.7	Inspecting Array Structure	19
2	Indexing and Slicing	20
2.1	Indexing One-Dimensional Arrays	20
2.2	Indexing Multidimensional Arrays	21
3	Slicing	23
3.1	Slicing Multidimensional Arrays	24
3.2	Dimensionality After Indexing	26
4	Boolean Indexing and Filtering	27
4.1	Combining Conditions	28

4.2	Filtering Multidimensional Data	29
5	Reshaping Arrays	30
5.1	Inferring a Dimension	32
5.2	Flattening an Array	32
6	Copies and Views	33
6.1	Views from Slicing	34
6.2	Explicit Copies	35
7	Vectorized Operations	36
7.1	Vectorization	38
8	Broadcasting	39
8.1	Broadcasting Across a Matrix	40
8.2	Broadcasting Rules	41
8.3	Row and Column Broadcasting	42
8.4	Broadcasting in Data Preprocessing	44
9	Universal Functions	45
9.1	Arithmetic Universal Functions	47
9.2	Mathematical Universal Functions	48
9.3	Comparison Universal Functions	50
9.4	Combining Universal Functions	51
10	Aggregations	52
11	Axis Semantics	54
11.1	Aggregating Along axis=0	55
11.2	Aggregating Along axis=1	56
11.3	Thinking About Axes	56
11.4	Preserving Reduced Dimensions	57
12	Cumulative Operations	58
13	Data Types and Numerical Behaviour	59
13.1	Memory Consumption	60
13.2	Integer Range	61
13.3	Floating-Point Precision	61
13.4	Type Conversion	62
14	Random Number Generation	63
14.1	Random Number Generators	63
14.2	Reproducibility and Seeds	64
14.3	Sampling from a Distribution	65
14.4	Random Choice	70
14.5	Sampling With and Without Replacement	71
14.6	Permutations and Shuffling	72

15 Arrays in Data Science and Machine Learning	73
15.1 Feature Transformation	74
15.2 Linear Models	75
15.3 Model Evaluation	75
15.4 Shape as Part of the Interface	76
16 Key Takeaways	78
17 Problems	79
17.1 Problem 1 — Creating and Inspecting Arrays	79
17.2 Problem 2 — One-Dimensional and Two-Dimensional Arrays	79
17.3 Problem 3 — Array Creation Functions	80
17.4 Problem 4 — Indexing	80
17.5 Problem 5 — Slicing	81
17.6 Problem 6 — Shape Preservation	82
17.7 Problem 7 — Boolean Masks	82
17.8 Problem 8 — Filtering Observations	83
17.9 Problem 9 — Reshaping	83
17.10 Problem 10 — Inferring Dimensions	84
17.11 Problem 11 — Flattening	84
17.12 Problem 12 — Assignment, View, or Copy?	85
17.13 Problem 13 — Why Views Matter	85
17.14 Problem 14 — Element-Wise Operations	85
17.15 Problem 15 — Element-Wise Versus Matrix Multiplication	86
17.16 Problem 16 — Vectorization	86
17.17 Problem 17 — Broadcasting with a Scalar	87
17.18 Problem 18 — Broadcasting Across Rows	87
17.19 Problem 19 — Broadcasting Rules	88
17.20 Problem 20 — Row and Column Broadcasting	89
17.21 Problem 21 — Universal Functions	89
17.22 Problem 22 — Aggregations	90
17.23 Problem 23 — Reasoning About axis	91
17.24 Problem 24 — keepdims	91
17.25 Problem 25 — Population and Sample Standard Deviation	92
17.26 Problem 26 — Data Types and Memory	92
17.27 Problem 27 — Integer Representation	93
17.28 Problem 28 — Floating-Point Comparison	93
17.29 Problem 29 — Type Conversion	93
17.30 Problem 30 — Reproducible Random Numbers	94
17.31 Problem 31 — Sampling from a Normal Distribution	94
17.32 Problem 32 — Sampling with and without Replacement	95
17.33 Problem 33 — Feature Centering	95
17.34 Problem 34 — Feature Standardization	96
17.35 Problem 35 — Linear Prediction with Arrays	96
17.36 Problem 36 — Residuals and Mean Squared Error	97
17.37 Problem 37 — Debugging a Shape Error	98
17.38 Problem 38 — Broadcasting Bug	98

17.39	Problem 39 — Diagnosing Unexpected Mutation	99
17.40	Problem 40 — Integrated Array Reasoning	99
17.41	Problem 41 — Designing an Array-Based Preprocessing Function	100
17.42	Problem 42 — Reasoning About an Unfamiliar Numerical Pipeline	101

1 Arrays

Numerical computing frequently requires performing the same operations over collections of values. In data science and machine learning, these collections may represent observations, features, model parameters, predictions, images, time series, or intermediate numerical results.

Python's built-in data structures, particularly lists, can represent collections of numeric values:

```
1 values = [1.0, 2.0, 3.0, 4.0]
```

However, general-purpose Python lists are not designed specifically for large-scale numerical computation. Their flexibility allows them to contain objects of different types:

```
1 values = [1, 2.5, "three", True]
```

Numerical algorithms usually benefit from a more specialized representation in which values have a consistent data type and operations can be performed efficiently over entire collections.

This is the role of **arrays**.

In these notes, the term *array* refers primarily to NumPy's `ndarray`, which is the standard array representation used throughout the scientific Python ecosystem.

1.1 NumPy and the `ndarray`

NumPy (*Numerical Python*) is a Python library designed for numerical computing. Its central data structure is the **N-dimensional array**, represented by the `ndarray` class.

By convention, NumPy is imported as:

```
1 import numpy as np
```

A simple one-dimensional array can be created from a Python list:

```
1 values = np.array([1, 2, 3, 4, 5])
2
3 values
```

```
array([1, 2, 3, 4, 5])
```

The resulting object is not a Python list:

```
1 type(values)
```

```
numpy.ndarray
```

It is an instance of `numpy.ndarray`.

The name `ndarray` stands for **N-dimensional array**. The same data structure can represent a scalar-like zero-dimensional array, a vector-like one-dimensional array, a matrix-like two-dimensional array, or an array with any higher number of dimensions.

For example:

```
1 vector = np.array([1, 2, 3])
2
3 matrix = np.array([
4     [1, 2, 3],
5     [4, 5, 6]
6 ])
```

Although terms such as *vector* and *matrix* are useful mathematical interpretations, NumPy represents both objects using the same underlying `ndarray` abstraction.

This is important in machine learning, where the shape of an array often gives it its meaning. A two-dimensional array with shape

$$(n, p)$$

can naturally represent a dataset containing (n) observations and (p) features.

In this interpretation:

- rows correspond to observations;
- columns correspond to features.

For example:

```
1 X = np.array([
2     [1.2, 3.4, 5.1],
3     [2.1, 2.8, 4.7],
4     [0.9, 3.7, 5.5],
5     [1.8, 3.1, 4.9]
6 ])
7
8 X.shape
```

(4, 3)

The shape (4, 3) indicates four observations and three features.

The array abstraction therefore provides more than a container for numbers. It provides a structured representation in which **dimensionality, shape, data type, and numerical operations** are explicitly defined.

1.2 Creating Arrays

The most direct way to create a NumPy array is with `np.array()`.

1.2.1 Creating Arrays from Python Sequences

A Python list can be converted into an array:

```
1 a = np.array([1, 2, 3, 4, 5])
2
3 a
```

array([1, 2, 3, 4, 5])

A tuple can be used as well:

```
1 b = np.array((10, 20, 30, 40))
2
3 b
```

array([10, 20, 30, 40])

Nested sequences produce multidimensional arrays:

```
1 matrix = np.array([
2     [1, 2, 3],
3     [4, 5, 6]
4 ])
5
6 matrix
```

```
array([[1, 2, 3],
       [4, 5, 6]])
```

The nested structure determines the dimensions and shape of the resulting array.

For regular numerical arrays, the nested sequences should have compatible lengths. Conceptually, a two-dimensional array represents a rectangular structure rather than an arbitrary collection of differently sized rows.

1.2.2 Homogeneous Data Types

Unlike a general Python list, a NumPy array normally stores elements using a single **data type**.

Consider:

```
1 integers = np.array([1, 2, 3, 4])
2
3 integers.dtype
```

```
dtype('int64')
```

NumPy determines an appropriate integer data type for the array.

If integers and floating-point values are combined:

```
1 values = np.array([1, 2.5, 3, 4])
2
3 values
```

```
array([1. , 2.5, 3. , 4. ])
```

```
1 values.dtype
```

```
dtype('float64')
```

NumPy chooses a common type capable of representing the values. In this case, the integer values are represented as floating-point numbers as part of the same array.

This differs fundamentally from a Python list:

```
1 values_list = [1, 2.5, 3, 4]
2
3 [type(value) for value in values_list]
```

```
[int, float, int, int]
```

The list stores references to individual Python objects, which may have different types. A NumPy array instead provides a homogeneous numerical representation.

This homogeneous representation is one of the properties that makes NumPy arrays suitable for efficient numerical computation.

1.3 Dimensions

The **dimension**, or number of axes, of an array describes how many indices are required to identify an element within its structure.

NumPy exposes this information through the `ndim` attribute.

1.3.1 Zero-Dimensional Arrays

A scalar value can be represented as a zero-dimensional array:

```
1 a = np.array(42)
2
3 a
```

```
array(42)
```



```
1 a.ndim
```

```
0
```

The result is 0.

Although the array contains a value, there is no axis along which elements are arranged.

1.3.2 One-Dimensional Arrays

A sequence of values produces a one-dimensional array:

```
1 a = np.array([1, 2, 3, 4, 5])
```

```
2
```

```
3 a.ndim
```

```
1
```

This structure has one axis.

It is often interpreted as a **vector**, although NumPy itself does not distinguish between row vectors and column vectors at this dimensionality.

1.3.3 Two-Dimensional Arrays

Nested sequences can produce a two-dimensional array:

```
1 A = np.array([
```

```
2     [1, 2, 3],
```

```
3     [4, 5, 6]
```

```
4 ])
```

```
5
```

```
6 A.ndim
```

```
2
```

A two-dimensional array has two axes and can naturally be interpreted as a matrix.

For the array above, an element requires two indices:

$$A_{ij},$$

where (i) identifies a row and (j) identifies a column.

1.3.4 Higher-Dimensional Arrays

NumPy arrays are not restricted to vectors and matrices.

For example:

```
1 A = np.array([
2     [
3         [1, 2],
4         [3, 4]
5     ],
6     [
7         [5, 6],
8         [7, 8]
9     ]
10 ])
11
12 A.ndim
```

3

This array has three dimensions.

Higher-dimensional arrays arise naturally in machine learning. For example, a collection of RGB images may be represented conceptually with dimensions corresponding to

(samples, height, width, channels).

The number of dimensions therefore reflects the structure of the data rather than merely its total number of values.

1.4 Shape

While `ndim` tells us **how many axes** an array has, `shape` tells us **how many elements exist along each axis**.

Consider:

```
1 a = np.array([10, 20, 30, 40, 50])
2
3 a.shape
```

(5,)

The result is:

```
1 (5,)
```

This notation indicates a one-dimensional array containing five elements.

The trailing comma is significant in Python: (5,) represents a tuple with one element.

For a two-dimensional array:

```
1 A = np.array([
2     [1, 2, 3],
3     [4, 5, 6]
4 ])
5
6 A.shape
```

(2, 3)

the result is:

```
1 (2, 3)
```

The array has:

- 2 elements along axis 0;
- 3 elements along axis 1.

When interpreted as a matrix, this corresponds to 2 rows and 3 columns.

For a three-dimensional array:

```
1 A = np.array([
2     [
3         [1, 2, 3],
4         [4, 5, 6]
5     ],
6     [
7         [7, 8, 9],
8         [10, 11, 12]
9     ]
10 ])
```

```
9     ]
10  ])
11
12  A.shape
```

(2, 2, 3)

the result is:

```
1  (2, 2, 3)
```

The array therefore has three axes whose lengths are 2, 2, and 3.

1.4.1 Shape as Structural Information

The shape of an array is fundamental in numerical computing because it determines which operations are structurally compatible.

For example, consider a typical machine learning feature matrix:

```
1  X = np.array([
2      [1.2, 2.3, 3.1],
3      [1.8, 2.7, 3.5],
4      [0.9, 2.1, 2.8],
5      [1.5, 2.5, 3.2]
6  ])
7
8  X.shape
```

(4, 3)

If `X.shape` is (4, 3), then under the usual machine learning convention:

$$X \in \mathbb{R}^{4 \times 3},$$

meaning that the dataset contains four observations and three features.

A corresponding target array might be:

```
1 y = np.array([0, 1, 0, 1])
2
3 y.shape
```

(4,)

Its shape is (4,), indicating one target value for each of the four observations.

Reasoning about shapes is therefore a central skill when working with numerical and machine learning code.

1.5 Size

The size attribute gives the total number of elements contained in an array.

For example:

```
1 A = np.array([
2     [1, 2, 3],
3     [4, 5, 6]
4 ])
5
6 A.size
```

6

The array has shape (2, 3), so its total number of elements is

$$2 \times 3 = 6.$$

More generally, if an array has shape

$$(d_1, d_2, \dots, d_k),$$

then its total size is

$$N = \prod_{i=1}^k d_i. \quad (1)$$

For example:

```
1 A = np.zeros((2, 3, 4))
2
3 print(A.shape)
4 print(A.size)
```

```
(2, 3, 4)
24
```

The size is

$$2 \times 3 \times 4 = 24.$$

The concepts should therefore be distinguished carefully:

- `ndim` gives the number of axes;
- `shape` gives the length of each axis;
- `size` gives the total number of elements.

1.6 Data Type

Every NumPy array has an associated data type, or **dtype**, accessible through the `dtype` attribute.

For example:

```
1 integers = np.array([1, 2, 3, 4])
2
3 integers.dtype
```

```
dtype('int64')
```

and:

```
1 floating_values = np.array([1.0, 2.0, 3.0, 4.0])
2
3 floating_values.dtype
```

```
dtype('float64')
```

The exact representation may depend on the platform and NumPy configuration, but the distinction between integer and floating-point storage remains important.

A data type can also be specified explicitly when the array is created:

```
1 values = np.array([1, 2, 3, 4], dtype=float)
2
3 values
```

```
array([1., 2., 3., 4.])
```

```
1 values.dtype
```

```
dtype('float64')
```

This converts the values to an appropriate floating-point representation.

Similarly:

```
1 values = np.array([1.2, 2.8, 3.4], dtype=int)
2
3 values
```

```
array([1, 2, 3])
```

Converting floating-point values to an integer dtype removes their fractional components. Type conversion can therefore alter information and should be performed deliberately.

The dtype of an array affects:

- how values are represented in memory;
- the range and precision of representable values;
- memory consumption;
- the behaviour of some numerical operations.

These considerations become particularly important when working with large datasets, numerical precision, and machine learning frameworks.

1.7 Creating Structured Arrays

Arrays do not always need to be constructed from existing Python sequences. NumPy provides functions for creating arrays with particular structures directly.

1.7.1 Arrays of Zeros

`np.zeros()` creates an array filled with zeros:

```
1 np.zeros(5)
```

```
array([0., 0., 0., 0., 0.])
```

A multidimensional shape can be supplied as a tuple:

```
1 np.zeros((3, 4))
```

```
array([[0., 0., 0., 0.],  
       [0., 0., 0., 0.],  
       [0., 0., 0., 0.]])
```

By default, the resulting values are floating-point numbers.

A different dtype can be requested explicitly:

```
1 np.zeros((3, 4), dtype=int)
```

```
array([[0, 0, 0, 0],  
       [0, 0, 0, 0],  
       [0, 0, 0, 0]])
```

Arrays of zeros are commonly used to initialize storage for numerical results.

1.7.2 Arrays of Ones

Similarly, `np.ones()` creates arrays filled with ones:

```
1 np.ones(5)
```

```
array([1., 1., 1., 1., 1.])
```

or:


```
1 np.ones((2, 3))
```

```
array([[1., 1., 1.],  
       [1., 1., 1.]])
```

1.7.3 Arrays Filled with a Constant

`np.full()` creates an array with a specified shape and fills it with a chosen value:

```
1 np.full((2, 4), 7)
```

```
array([[7, 7, 7, 7],  
       [7, 7, 7, 7]])
```

This can be useful when a computation requires a particular initial value rather than zero or one.

1.7.4 Numerical Sequences with `arange()`

`np.arange()` creates regularly spaced values and behaves similarly to Python's `range()`:

```
1 np.arange(5)
```

```
array([0, 1, 2, 3, 4])
```

A starting point and endpoint can be specified:

```
1 np.arange(2, 10)
```

```
array([2, 3, 4, 5, 6, 7, 8, 9])
```

as well as a step:

```
1 np.arange(2, 10, 2)
```

```
array([2, 4, 6, 8])
```

As with `range()`, the endpoint is excluded.

`arange()` can also produce floating-point sequences:

```
1 np.arange(0, 1, 0.2)
```

```
array([0. , 0.2, 0.4, 0.6, 0.8])
```

However, floating-point steps can interact with finite-precision representation. When a specific number of evenly spaced points over an interval is required, `linspace()` is often a clearer choice.

1.7.5 Evenly Spaced Values with `linspace()`

`np.linspace()` creates a specified number of evenly spaced values over an interval.

For example:

```
1 np.linspace(0, 1, 5)
```

```
array([0. , 0.25, 0.5 , 0.75, 1.  ])
```

By default, both endpoints are included.

Mathematically, for an interval $([a,b])$ divided into (n) equally spaced values, the spacing is

$$\Delta x = \frac{b - a}{n - 1}. \quad (2)$$

For example:

```
1 x = np.linspace(0, 10, 6)
```

```
2
```

```
3 x
```

```
array([ 0.,  2.,  4.,  6.,  8., 10.])
```

produces six values spanning the interval from 0 to 10.

This distinction between `arange()` and `linspace()` is useful:

- `arange()` is naturally specified in terms of a **step size**;
- `linspace()` is naturally specified in terms of a **number of points**.

For numerical work over continuous intervals, `linspace()` is often particularly convenient.

1.7.6 Identity Matrices

NumPy can construct an identity matrix using `np.eye()`:

```
1 I = np.eye(4)
2
3 I
```

```
array([[1., 0., 0., 0.],
       [0., 1., 0., 0.],
       [0., 0., 1., 0.],
       [0., 0., 0., 1.]])
```

The identity matrix (I_n) is defined by

$$(I_n)_{ij} = \begin{cases} 1, & i = j, \\ 0, & i \neq j. \end{cases} \quad (3)$$

Identity matrices arise frequently in linear algebra, statistical modelling, optimization, and numerical algorithms.

1.7.7 Inspecting Array Structure

Several structural properties can be inspected together:

```
1 A = np.zeros((3, 4), dtype=float)
2
3 print("Number of dimensions:", A.ndim)
4 print("Shape:", A.shape)
5 print("Size:", A.size)
6 print("Data type:", A.dtype)
```

```
Number of dimensions: 2
Shape: (3, 4)
Size: 12
Data type: float64
```

These four attributes provide a concise description of an array:

- `ndim`: how many axes the array has;
- `shape`: the length of each axis;

- size: the total number of elements;
- dtype: how those elements are represented.

When working with unfamiliar numerical code, inspecting these properties is often one of the first steps in understanding the data being processed.

2 Indexing and Slicing

Once an array has been created, individual elements or subsets of elements can be accessed through **indexing** and **slicing**.

These operations are fundamental in numerical computing because datasets are rarely processed only as complete objects. We frequently need to select observations, extract features, isolate regions of an image, or operate on particular dimensions of a larger array.

2.1 Indexing One-Dimensional Arrays

Indexing a one-dimensional NumPy array closely resembles indexing a Python list.

Consider:

```
1 a = np.array([10, 20, 30, 40, 50])
```

The first element is accessed with index 0:

```
1 a[0]
```

```
np.int64(10)
```

The third element is:

```
1 a[2]
```

```
np.int64(30)
```

Python uses **zero-based indexing**, so an array containing (n) elements has valid positive indices from (0) through (n-1).

Negative indices count backward from the end:

```
1 a[-1]
```

```
np.int64(50)
```

```
1 a[-2]
```

```
np.int64(40)
```

Thus, `a[-1]` refers to the last element and `a[-2]` to the second-to-last element.

2.2 Indexing Multidimensional Arrays

For a two-dimensional array, an element can be selected by providing one index for each axis.

Consider:

```
1 A = np.array([
2     [10, 20, 30],
3     [40, 50, 60],
4     [70, 80, 90]
5 ])
6
7 A
```

```
array([[10, 20, 30],
       [40, 50, 60],
       [70, 80, 90]])
```

The element in the first row and second column is:

```
1 A[0, 1]
```

```
np.int64(20)
```

Similarly:

```
1 A[2, 0]
```

```
np.int64(70)
```

selects the element in the third row and first column.

The notation

```
1 A[i, j]
```

corresponds naturally to the mathematical notation

$$A_{ij},$$

with the important difference that NumPy indices begin at zero.

It is also possible to write:

```
1 A[0][1]
```

`np.int64(20)`

However,

```
1 A[0, 1]
```

is generally preferable because it expresses multidimensional indexing directly rather than performing two successive indexing operations.

For arrays with more dimensions, one index can be supplied for each axis:

```
1 B = np.arange(24).reshape(2, 3, 4)
2
3 B
```

```
array([[[ 0, 1, 2, 3],
        [ 4, 5, 6, 7],
        [ 8, 9, 10, 11]],
       [[12, 13, 14, 15],
        [16, 17, 18, 19],
        [20, 21, 22, 23]]])
```

```
1 B[1, 2, 3]
```

`np.int64(23)`

The number of indices required to select a single scalar element corresponds to the number of dimensions of the array.

3 Slicing

Slicing selects a range of elements rather than a single element.

The general syntax is:

```
1 array[start:stop:step]
```

where:

- start is the first included index;
- stop is the first excluded index;
- step determines the spacing between selected indices.

As with Python sequences, the stop position is not included.

Consider:

```
1 a = np.array([10, 20, 30, 40, 50, 60])
```

The slice

```
1 a[1:4]
```

```
array([20, 30, 40])
```

selects the elements at indices 1, 2, and 3.

Omitting start begins at the start of the array:

```
1 a[:3]
```

```
array([10, 20, 30])
```

Omitting stop continues to the end:

```
1 a[3:]
```

```
array([40, 50, 60])
```

A step can also be specified:

```
1 a[::2]
```

```
array([10, 30, 50])
```

Negative steps can reverse the direction:

```
1 a[::-1]
```

```
array([60, 50, 40, 30, 20, 10])
```

3.1 Slicing Multidimensional Arrays

Slicing can be applied independently along each axis.

Consider:

```
1 A = np.array([
2     [1, 2, 3, 4],
3     [5, 6, 7, 8],
4     [9, 10, 11, 12]
5 ])
6
7 A
```

```
array([[ 1,  2,  3,  4],
       [ 5,  6,  7,  8],
       [ 9, 10, 11, 12]])
```

To select the first two rows:

```
1 A[:2, :]
```

```
array([[1, 2, 3, 4],
       [5, 6, 7, 8]])
```

The colon : means that all elements along that axis should be selected.

To select the second and third columns:


```
1 A[:, 1:3]
```

```
array([[ 2,  3],  
       [ 6,  7],  
       [10, 11]])
```

To select the first two rows and the last two columns:

```
1 A[:, 2:]
```

```
array([[3, 4],  
       [7, 8]])
```

This notation becomes particularly important in machine learning.

Suppose X represents a feature matrix with shape

$$(n, p),$$

where rows correspond to observations and columns correspond to features.

Then:

```
1 X[:100, :]
```

selects the first 100 observations, while

```
1 X[:, :3]
```

selects the first three features for every observation.

The expression

```
1 X[i, j]
```

therefore has a structural interpretation:

- the first index operates along axis 0;
- the second index operates along axis 1.

Understanding this relationship between indexing and axes is essential when working with multidimensional numerical data.

3.2 Dimensionality After Indexing

An important distinction exists between selecting an element along an axis and selecting a slice of that axis.

Consider:

```
1 A = np.array([
2     [1, 2, 3],
3     [4, 5, 6]
4 ])
```

Selecting the first row using an integer index:

```
1 row = A[0]
2
3 print(row)
4 print(row.shape)
```

```
[1 2 3]
(3,)
```

produces a one-dimensional array with shape:

```
1 (3,)
```

By contrast:

```
1 row = A[0:1]
2
3 print(row)
4 print(row.shape)
```

```
[[1 2 3]]
(1, 3)
```

produces a two-dimensional array with shape:

```
1 (1, 3)
```

The values are closely related, but their shapes are different.

This distinction can be important when subsequent operations expect a particular number of dimensions.

For example, in machine learning code:

```
1 X[0]
```

often represents one observation as a one-dimensional feature vector, whereas:

```
1 X[0:1]
```

preserves the two-dimensional structure of a feature matrix containing one observation.

Shape should therefore be considered part of the meaning of an array, not merely a formatting detail.

4 Boolean Indexing and Filtering

NumPy arrays can be indexed using arrays of boolean values.

This provides a concise and powerful mechanism for filtering numerical data.

Consider:

```
1 values = np.array([12, 5, 18, 7, 21, 3])
```

A comparison applied to the entire array produces a boolean array:

```
1 values > 10
```

```
array([ True, False,  True, False,  True, False])
```

Conceptually, NumPy evaluates the condition element by element:

$$m_i = \begin{cases} \text{True}, & x_i > 10, \\ \text{False}, & x_i \leq 10. \end{cases}$$

The resulting boolean array is often called a **mask**.

It can be used directly to select elements:

```
1 mask = values > 10
2
3 values[mask]
```

```
array([12, 18, 21])
```

The operation can also be written more compactly:

```
1 values[values > 10]
```

```
array([12, 18, 21])
```

Only elements for which the corresponding condition is True are returned.

4.1 Combining Conditions

Multiple boolean conditions can be combined.

For example:

```
1 values[(values >= 5) & (values <= 18)]
```

```
array([12, 5, 18, 7])
```

The operator & performs element-wise logical AND.

Similarly, | performs element-wise logical OR:

```
1 values[(values < 5) | (values > 18)]
```

```
array([21, 3])
```

and ~ negates a boolean mask:

```
1 mask = values > 10
2
3 values[~mask]
```

```
array([5, 7, 3])
```

Parentheses around individual comparisons are important because of Python's operator precedence rules.

For NumPy arrays, expressions such as

```
1 (values >= 5) & (values <= 18)
```

should be used rather than Python's scalar logical form:

```
1 values >= 5 and values <= 18
```

The operators `and` and `or` evaluate the truth value of complete Python objects, whereas `&` and `|` operate element by element on boolean arrays.

4.2 Filtering Multidimensional Data

Boolean indexing becomes especially useful when rows represent observations.

Consider:

```
1 X = np.array([
2     [25, 50000],
3     [42, 72000],
4     [31, 61000],
5     [55, 90000],
6     [28, 48000]
7 ])
```

Suppose the first column represents age and the second represents income.

We can identify observations with age greater than 30:

```
1 mask = X[:, 0] > 30
2
3 mask
```

```
array([False,  True,  True,  True, False])
```

and use the mask to select complete rows:

```
1 X[mask]
```

```
array([[ 42, 72000],  
       [ 31, 61000],  
       [ 55, 90000]])
```

A condition based on one feature can therefore determine which observations are retained from the entire feature matrix.

This pattern occurs frequently during exploratory data analysis, preprocessing, and validation.

5 Reshaping Arrays

The same collection of values can often be interpreted using different array shapes.

NumPy's `reshape()` operation changes the shape of an array without changing its number of elements.

Consider:

```
1 a = np.arange(12)  
2  
3 a
```

```
array([ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11])
```

Its shape is:

```
1 a.shape
```

```
(12,)
```

The 12 values can be reorganized into a (3×4) array:

```
1 A = a.reshape(3, 4)  
2  
3 A
```

```
array([[ 0, 1, 2, 3],  
       [ 4, 5, 6, 7],  
       [ 8, 9,10,11]])
```

```
1 A.shape
```

```
(3, 4)
```

The original and reshaped arrays contain the same number of elements:

$$12 = 3 \times 4.$$

Consequently, the following reshape is not possible:

```
1 a.reshape(5, 3)
```

because

$$12 \neq 5 \times 3.$$

More generally, if an array has (N) elements, a reshape from dimensions

$$(d_1, d_2, \dots, d_k)$$

to

$$(e_1, e_2, \dots, e_m)$$

requires

$$\prod_{i=1}^k d_i = \prod_{j=1}^m e_j = N. \quad (4)$$

Reshaping changes the structural interpretation of the data, but it cannot create or remove elements.

5.1 Inferring a Dimension

One dimension can be specified as -1, allowing NumPy to infer its size.

For example:

```
1 a = np.arange(12)
2
3 a.reshape(3, -1)
```

```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

Since the array contains 12 elements and the first dimension is 3, NumPy infers:

$$\frac{12}{3} = 4.$$

The resulting shape is therefore (3, 4).

Similarly:

```
1 a.reshape(-1, 2)
```

```
array([[ 0,  1],
       [ 2,  3],
       [ 4,  5],
       [ 6,  7],
       [ 8,  9],
       [10, 11]])
```

produces shape (6, 2).

Only one dimension can be inferred in this way because the remaining dimensions must provide enough information to determine it uniquely.

5.2 Flattening an Array

A multidimensional array can be converted into a one-dimensional representation.

For example:


```
1 A = np.array([
2     [1, 2, 3],
3     [4, 5, 6]
4 ])
5
6 A.reshape(-1)
```

```
array([1, 2, 3, 4, 5, 6])
```

NumPy also provides methods such as `flatten()` and `ravel()`:

```
1 A.flatten()
```

```
array([1, 2, 3, 4, 5, 6])
```

```
1 A.ravel()
```

```
array([1, 2, 3, 4, 5, 6])
```

Although these operations can produce the same visible values, their memory behaviour is not necessarily identical.

In particular, `flatten()` returns a copy, whereas `ravel()` returns a view whenever possible.

This distinction leads to an important aspect of NumPy arrays: the relationship between an array's logical structure and the underlying memory in which its values are stored.

6 Copies and Views

When working with Python objects, assignment does not generally create an independent copy.

For example:

```
1 a = np.array([1, 2, 3, 4])
2
3 b = a
```

Both names refer to the same array object:

```

1 b[0] = 100
2
3 print(a)
4 print(b)

```

```

[100  2  3  4]
[100  2  3  4]

```

Changing the array through b is therefore visible through a.

NumPy introduces an additional concept: a new array object can sometimes share the same underlying data with another array.

Such an object is called a **view**.

6.1 Views from Slicing

Basic slicing generally produces a view rather than an independent copy.

Consider:

```

1 a = np.array([10, 20, 30, 40, 50])
2
3 b = a[1:4]
4
5 b

```

```
array([20, 30, 40])
```

If the slice is modified:

```

1 b[0] = 999
2
3 print(b)
4 print(a)

```

```

[999 30 40]
[ 10 999 30 40 50]

```

the original array also changes.

The objects `a` and `b` are distinct array objects, but their data are not completely independent. The slice refers to part of the same underlying memory.

This behaviour differs from ordinary Python list slicing, which creates a new list.

Views allow NumPy to manipulate large arrays efficiently because many operations can avoid copying potentially large blocks of numerical data.

However, they also mean that modifications can propagate in ways that may be unexpected if memory sharing is not understood.

6.2 Explicit Copies

An independent copy can be created with `copy()`:

```
1 a = np.array([10, 20, 30, 40, 50])
2
3 b = a[1:4].copy()
4
5 b[0] = 999
6
7 print(b)
8 print(a)
```

```
[999 30 40]
[10 20 30 40 50]
```

Now modifying `b` does not alter `a`.

The distinction can be summarized as:

- **assignment** may create another name for the same array object;
- a **view** creates another array object that shares underlying data;
- a **copy** creates an array with independent data.

This distinction is particularly important with large datasets. Copies provide independence but require additional memory and computation, while views can be substantially more efficient.

The appropriate choice depends on whether shared modification is intended.

7 Vectorized Operations

One of the most important features of NumPy is the ability to perform operations on complete arrays without explicitly writing Python loops.

Consider two arrays:

```
1 a = np.array([1, 2, 3, 4])
2 b = np.array([10, 20, 30, 40])
```

Addition can be performed directly:

```
1 a + b
```

```
array([11, 22, 33, 44])
```

The operation is applied element by element:

$$\mathbf{a} + \mathbf{b} = \begin{bmatrix} a_1 + b_1 \\ a_2 + b_2 \\ \vdots \\ a_n + b_n \end{bmatrix}.$$

Similarly:

```
1 a - b
```

```
array([-9, -18, -27, -36])
```

```
1 a * b
```

```
array([ 10,  40,  90, 160])
```

```
1 a / b
```

```
array([0.1, 0.1, 0.1, 0.1])
```

```
1 a ** 2
```

```
array([ 1,  4,  9, 16])
```

These are **element-wise operations**.

It is important not to confuse element-wise multiplication with matrix multiplication.

For two compatible matrices:

```
1 A = np.array([
2     [1, 2],
3     [3, 4]
4 ])
5
6 B = np.array([
7     [5, 6],
8     [7, 8]
9 ])
```

the expression

```
1 A * B
```

```
array([[ 5, 12],
       [21, 32]])
```

multiplies corresponding elements.

Matrix multiplication instead uses the @ operator:

```
1 A @ B
```

```
array([[19, 22],
       [43, 50]])
```

Mathematically, element-wise multiplication gives

$$(A \odot B)_{ij} = A_{ij}B_{ij}, \quad (5)$$

whereas matrix multiplication gives

$$(AB)_{ij} = \sum_k A_{ik}B_{kj}. \quad (6)$$

The distinction is fundamental in numerical computing and machine learning.

7.1 Vectorization

Expressing computations directly as array operations is commonly called **vectorization**.

Suppose we want to transform each value according to

$$y_i = 2x_i + 3.$$

Using an explicit Python loop:

```
1 x = np.array([1, 2, 3, 4, 5])
2
3 result = []
4
5 for value in x:
6     result.append(2 * value + 3)
7
8 result
```

```
[np.int64(5), np.int64(7), np.int64(9), np.int64(11), np.int64(13)]
```

The vectorized NumPy form is:

```
1 2 * x + 3
```

```
array([ 5, 7, 9, 11, 13])
```

The second expression describes the numerical transformation directly.

Vectorized code is often:

- more concise;
- closer to the underlying mathematical notation;
- easier to compose;
- computationally more efficient than explicit Python-level loops.

The performance advantage arises because NumPy can perform many numerical operations in optimized compiled code rather than repeatedly executing Python instructions for individual elements.

Vectorization should not be understood merely as “writing shorter code.” It represents a different level of abstraction: operations are expressed over arrays rather than individual scalar elements.

This perspective is central to numerical computing and carries directly into machine learning frameworks.

8 Broadcasting

Element-wise operations are straightforward when arrays have identical shapes:

```
1 a = np.array([1, 2, 3])
2 b = np.array([10, 20, 30])
3
4 a + b
```

```
array([11, 22, 33])
```

However, NumPy can also perform many operations between arrays with different shapes.

This mechanism is called **broadcasting**.

The simplest example is an operation between an array and a scalar:

```
1 a = np.array([1, 2, 3, 4])
2
3 a + 10
```

```
array([11, 12, 13, 14])
```

Conceptually, the scalar behaves as if it had been expanded to match the array:

$$\begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix} + 10 \longrightarrow \begin{bmatrix} 1 + 10 \\ 2 + 10 \\ 3 + 10 \\ 4 + 10 \end{bmatrix}.$$

NumPy does not generally need to create a physical array containing four copies of 10. Broadcasting describes how the operation can be performed as though compatible expansion had occurred.

8.1 Broadcasting Across a Matrix

Consider:

```
1 X = np.array([
2     [1, 2, 3],
3     [4, 5, 6],
4     [7, 8, 9]
5 ])
6
7 offset = np.array([10, 20, 30])
```

Their shapes are:

```
1 print(X.shape)
2 print(offset.shape)
```

```
(3, 3)
(3,)
```

which are:

```
1 (3, 3)
2 (3,)
```

The operation

```
1 X + offset
```

```
array([[11, 22, 33],
       [14, 25, 36],
       [17, 28, 39]])
```

adds the vector to every row:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} + \begin{bmatrix} 10 & 20 & 30 \end{bmatrix} = \begin{bmatrix} 11 & 22 & 33 \\ 14 & 25 & 36 \\ 17 & 28 & 39 \end{bmatrix}.$$

This is possible because the trailing dimension of both arrays has size 3.

8.2 Broadcasting Rules

When NumPy compares two shapes for broadcasting, it compares their dimensions from **right to left**.

Two dimensions are compatible when:

1. they are equal; or
2. one of them is 1.

If one array has fewer dimensions, its shape can be conceptually padded with dimensions of size 1 on the left.

For example:

```
1 X:   (4, 3)
2 offset: (3,)
```

can be viewed conceptually as:

```
1 X:   (4, 3)
2 offset: (1, 3)
```

The dimensions are compatible:

```
1 3 and 3 -> equal
2 4 and 1 -> one is 1
```

The resulting shape is:

```
1 (4, 3)
```

Consider another example:

```
1 A:   (5, 4, 3)
2 B:   (4, 1)
```

Conceptually:

```
1 A:   (5, 4, 3)
2 B:   (1, 4, 1)
```

Comparing from right to left:

```
1 3 and 1 -> compatible
2 4 and 4 -> compatible
3 5 and 1 -> compatible
```

so the result has shape:

```
1 (5, 4, 3)
```

By contrast:

```
1 A: (4, 3)
2 B: (4,)
```

is not compatible because the trailing dimensions are:

```
1 3 and 4
```

and neither dimension is 1.

Understanding broadcasting is therefore largely a matter of **reasoning about shapes**.

8.3 Row and Column Broadcasting

A subtle but important distinction arises between arrays with shapes $(n,)$, $(1, n)$, and $(n, 1)$.

Consider:

```
1 row = np.array([1, 2, 3])
2
3 print(row.shape)
```

```
(3,)
```

The shape is:

```
1 (3,)
```

A two-dimensional row can instead be represented as:

```

1 row_2d = np.array([[1, 2, 3]])
2
3 print(row_2d.shape)

```

(1, 3)

with shape:

```

1 (1, 3)

```

A column can be represented as:

```

1 column = np.array([
2     [10],
3     [20],
4     [30]
5 ])
6
7 print(column.shape)

```

(3, 1)

with shape:

```

1 (3, 1)

```

Adding the row and column:

```

1 column + row

```

```

array([[11, 12, 13],
       [21, 22, 23],
       [31, 32, 33]])

```

uses broadcasting to produce a (3, 3) array:

$$\begin{bmatrix} 10 \\ 20 \\ 30 \end{bmatrix} + \begin{bmatrix} 1 & 2 & 3 \end{bmatrix} = \begin{bmatrix} 11 & 12 & 13 \\ 21 & 22 & 23 \\ 31 & 32 & 33 \end{bmatrix}.$$

This example demonstrates why dimensionality and shape must be tracked carefully. Arrays containing the same number of values can participate in very different operations depending on how those values are arranged across axes.

8.4 Broadcasting in Data Preprocessing

Broadcasting appears naturally in data science.

Suppose:

```
1 X = np.array([
2     [10.0, 100.0, 1000.0],
3     [12.0, 120.0, 1100.0],
4     [14.0, 140.0, 1200.0],
5     [16.0, 160.0, 1300.0]
6 ])
```

Each column represents a feature.

The mean of each feature can be calculated as:

```
1 feature_means = X.mean(axis=0)
2
3 feature_means
```

```
array([ 13., 130., 1150.])
```

The resulting shape is:

```
1 feature_means.shape
```

```
(3,)
```

Because `feature_means` has one value per column, NumPy can subtract it from every row:

```
1 X_centered = X - feature_means
2
3 X_centered
```

```
array([[ -3., -30., -150.],
       [ -1., -10., -50.],
       [  1.,  10.,  50.],
       [  3.,  30., 150.]])
```

Conceptually, for a feature matrix

$$X \in \mathbb{R}^{n \times p},$$

and a vector of feature means

$$\mu \in \mathbb{R}^p,$$

centering is

$$X_{\text{centered}} = X - \mu, \quad (7)$$

where broadcasting applies the subtraction to every observation.

For each element:

$$(X_{\text{centered}})_{ij} = X_{ij} - \mu_j. \quad (8)$$

This is an important example because it connects several NumPy concepts at once:

- the feature matrix has shape (n_samples, n_features);
- aggregation along axis=0 produces one statistic per feature;
- broadcasting applies those statistics across all observations;
- vectorization eliminates the need for explicit loops over observations or features.

The same pattern underlies many preprocessing operations used in machine learning, including feature centering and standardization.

Broadcasting is therefore not simply a convenient NumPy syntax. It is a fundamental mechanism for expressing numerical operations between arrays whose shapes have compatible mathematical roles.

9 Universal Functions

NumPy provides a large collection of functions designed to operate efficiently on arrays element by element.

These functions are called **universal functions**, or **ufuncs**.

For example:

```

1 x = np.array([1, 4, 9, 16])
2
3 np.sqrt(x)

```

```
array([1., 2., 3., 4.])
```

The square-root function is applied independently to each element:

$$y_i = \sqrt{x_i}.$$

Other common examples include:

```

1 x = np.array([1.0, 2.0, 3.0, 4.0])
2
3 np.exp(x)

```

```
array([ 2.71828183,  7.3890561 , 20.08553692, 54.59815003])
```

```
1 np.log(x)
```

```
array([0.         , 0.69314718, 1.09861229, 1.38629436])
```

```
1 np.sin(x)
```

```
array([ 0.84147098,  0.90929743,  0.14112001, -0.7568025 ])
```

```
1 np.abs(np.array([-3, -1, 0, 2]))
```

```
array([3, 1, 0, 2])
```

A ufunc therefore represents an operation defined at the scalar level but efficiently extended over complete arrays.

This is closely related to vectorization.

For example, the expression

```
1 np.sqrt(x)
```

```
array([1.        , 1.41421356, 1.73205081, 2.        ])
```

is conceptually equivalent to applying the scalar transformation

$$f(x) = \sqrt{x}$$

to every element of the array.

However, NumPy performs this operation internally without requiring an explicit Python loop.

9.1 Arithmetic Universal Functions

Many familiar arithmetic operations correspond to ufuncs.

Consider:

```
1 a = np.array([1, 2, 3, 4])
2 b = np.array([10, 20, 30, 40])
```

Addition can be written using the operator:

```
1 a + b
```

```
array([11, 22, 33, 44])
```

or explicitly using:

```
1 np.add(a, b)
```

```
array([11, 22, 33, 44])
```

Similarly:

```
1 np.subtract(a, b)
```

```
array([-9, -18, -27, -36])
```

```
1 np.multiply(a, b)
```

```
array([ 10, 40, 90, 160])
```

```
1 np.divide(a, b)
```

```
array([0.1, 0.1, 0.1, 0.1])
```

```
1 np.power(a, 2)
```

```
array([ 1, 4, 9, 16])
```

The operator syntax is generally more natural for ordinary arithmetic, while the explicit function form can be useful when working programmatically with function objects.

For example:

```
1 operation = np.multiply
```

```
2
```

```
3 operation(a, b)
```

```
array([ 10, 40, 90, 160])
```

This connects NumPy operations with the concept of first-class functions considered previously.

9.2 Mathematical Universal Functions

NumPy includes many commonly used mathematical transformations.

Examples include:

```
1 x = np.array([1.0, 2.0, 4.0, 8.0])
```

```
2
```

```
3 np.log(x)
```

```
array([0.         , 0.69314718, 1.38629436, 2.07944154])
```



```
1 np.log2(x)
```

```
array([0., 1., 2., 3.])
```

```
1 np.exp(x)
```

```
array([2.71828183e+00, 7.38905610e+00, 5.45981500e+01, 2.98095799e+03])
```

```
1 np.sqrt(x)
```

```
array([1.      , 1.41421356, 2.      , 2.82842712])
```

Trigonometric functions are also available:

```
1 angles = np.array([0, np.pi / 2, np.pi])
```

```
2
```

```
3 np.sin(angles)
```

```
array([0.0000000e+00, 1.0000000e+00, 1.2246468e-16])
```

```
1 np.cos(angles)
```

```
array([ 1.0000000e+00,  6.123234e-17, -1.0000000e+00])
```

These functions preserve the shape of the original array when applied element-wise.

If

$$X \in \mathbb{R}^{m \times n},$$

and f is an element-wise ufunc, then the resulting array has the same shape:

$$f(X) \in \mathbb{R}^{m \times n}.$$

This shape-preserving property makes ufuncs easy to compose into numerical transformations.

9.3 Comparison Universal Functions

Comparisons are vectorized as well.

For example:

```
1 x = np.array([1, 5, 2, 8, 3])
2
3 x > 3
```

```
array([False,  True, False,  True, False])
```

The result is a boolean array.

Equivalent NumPy functions include:

```
1 np.greater(x, 3)
```

```
array([False,  True, False,  True, False])
```

```
1 np.less(x, 3)
```

```
array([ True, False,  True, False, False])
```

```
1 np.equal(x, 3)
```

```
array([False, False, False, False,  True])
```

These operations are frequently combined with boolean indexing.

For example:

```
1 x[np.greater(x, 3)]
```

```
array([5, 8])
```

Thus, filtering can be understood as the combination of:

1. a vectorized comparison that creates a boolean mask;
2. indexing using that mask.

9.4 Combining Universal Functions

Because ufuncs generally operate element-wise and preserve compatible shapes, they can be composed naturally.

For example:

```
1 x = np.array([1.0, 2.0, 3.0, 4.0])
2
3 np.sqrt(np.exp(x))
```

```
array([1.64872127, 2.71828183, 4.48168907, 7.3890561 ])
```

Conceptually:

$$y_i = \sqrt{e^{x_i}}.$$

Similarly:

```
1 1 / (1 + np.exp(-x))
```

```
array([0.73105858, 0.88079708, 0.95257413, 0.98201379])
```

implements the logistic or sigmoid function:

$$\sigma(x) = \frac{1}{1 + e^{-x}}. \quad (9)$$

Applied to an array, the function becomes:

$$\sigma(\mathbf{x}) = \begin{bmatrix} \sigma(x_1) \\ \sigma(x_2) \\ \vdots \\ \sigma(x_n) \end{bmatrix}.$$

This is a simple example of how mathematical transformations used in machine learning can be expressed directly through vectorized NumPy operations.

10 Aggregations

While ufuncs often transform every element independently, **aggregation functions** summarize multiple array elements into fewer values.

Common aggregations include:

- sum;
- mean;
- minimum;
- maximum;
- variance;
- standard deviation.

Consider:

```
1 x = np.array([2, 4, 6, 8, 10])
```

The total is:

```
1 x.sum()
```

```
np.int64(30)
```

or equivalently:

```
1 np.sum(x)
```

```
np.int64(30)
```

The arithmetic mean is:

```
1 x.mean()
```

```
np.float64(6.0)
```

Other common examples are:

```
1 x.min()
```

```
np.int64(2)
```

```
1 x.max()
```

```
np.int64(10)
```

```
1 x.var()
```

```
np.float64(8.0)
```

```
1 x.std()
```

```
np.float64(2.8284271247461903)
```

These operations reduce multiple values to one summary statistic.

For a one-dimensional array

$$\mathbf{x} = (x_1, x_2, \dots, x_n),$$

the mean is:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i. \quad (10)$$

The default NumPy variance is the population variance:

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2. \quad (11)$$

If the sample variance is required, the divisor can be adjusted using `ddof=1`:

```
1 x.var(ddof=1)
```

```
np.float64(10.0)
```

Similarly:

```
1 x.std(ddof=1)
```

```
np.float64(3.1622776601683795)
```

uses the sample standard deviation.

This distinction is important in statistical work because NumPy's default convention is not necessarily the same convention required by a particular estimator.

11 Axis Semantics

Aggregations become particularly important in multidimensional arrays because we must specify **which axis is being reduced**.

Consider:

```
1 X = np.array([
2     [1, 2, 3],
3     [4, 5, 6],
4     [7, 8, 9],
5     [10, 11, 12]
6 ])
7
8 X.shape
```

(4, 3)

The shape is:

```
1 (4, 3)
```

Interpreted as a dataset, this could represent:

- 4 observations;
- 3 features.

If no axis is specified:

```
1 X.sum()
```

np.int64(78)

NumPy aggregates all elements and returns a scalar.

11.1 Aggregating Along axis=0

Consider:

```
1 X.sum(axis=0)
```

```
array([22, 26, 30])
```

The result contains one value per column.

Its shape is:

```
1 (3,)
```

Mathematically:

$$s_j = \sum_{i=1}^n X_{ij}.$$

The row dimension is reduced, while the feature dimension remains.

For a machine learning feature matrix:

```
1 X.mean(axis=0)
```

```
array([5.5, 6.5, 7.5])
```

therefore calculates one mean for each feature.

This is why feature centering can be written as:

```
1 feature_means = X.mean(axis=0)
2 X_centered = X - feature_means
```

11.2 Aggregating Along axis=1

Now consider:

```
1 X.sum(axis=1)
```

```
array([ 6, 15, 24, 33])
```

The result contains one value per row.

Its shape is:

```
1 (4,)
```

Mathematically:

$$s_i = \sum_{j=1}^p X_{ij}.$$

The feature dimension is reduced, while the observation dimension remains.

Thus:

```
1 X.mean(axis=1)
```

```
array([ 2.,  5.,  8., 11.])
```

computes one average for each observation.

11.3 Thinking About Axes

A useful way to reason about aggregation is:

The specified axis is the dimension that is collapsed by the operation.

For an array with shape:

```
1 (n_samples, n_features)
```

the operation:


```
1 X.mean(axis=0)
```

collapses the sample axis and leaves:

```
1 (n_features,)
```

whereas:

```
1 X.mean(axis=1)
```

collapses the feature axis and leaves:

```
1 (n_samples,)
```

This interpretation is often more reliable than memorizing statements such as “axis=0 means columns” or “axis=1 means rows.”

The axis parameter describes the dimension being **reduced**, not a particular visual direction.

This distinction becomes even more important in arrays with three or more dimensions.

11.4 Preserving Reduced Dimensions

By default, aggregation removes the reduced dimension.

For example:

```
1 feature_means = X.mean(axis=0)
2
3 feature_means.shape
```

(3,)

returns:

```
1 (3,)
```

The dimension can instead be preserved using `keepdims=True`:

```
1 feature_means = X.mean(axis=0, keepdims=True)
2
3 feature_means.shape
```

(1, 3)

The result now has shape:

```
1 (1, 3)
```

Similarly:

```
1 X.mean(axis=1, keepdims=True).shape
```

(4, 1)

produces:

```
1 (4, 1)
```

Preserving dimensions can be useful when subsequent broadcasting operations depend on explicit row or column structure.

12 Cumulative Operations

Some aggregations preserve the number of elements while accumulating information sequentially.

For example:

```
1 x = np.array([1, 2, 3, 4, 5])
2
3 np.cumsum(x)
```

array([1, 3, 6, 10, 15])

produces:

$$[1, 1 + 2, 1 + 2 + 3, \dots].$$

Similarly:

```
1 np.cumprod(x)
```

```
array([ 1,  2,  6, 24, 120])
```

calculates cumulative products.

For the (k)-th cumulative sum:

$$s_k = \sum_{i=1}^k x_i.$$

These operations are useful in contexts such as time series, cumulative returns, running totals, and sequential numerical analyses.

13 Data Types and Numerical Behaviour

The dtype of a NumPy array determines how its values are represented numerically.

Unlike Python's built-in int, which can represent arbitrarily large integers subject to available memory, NumPy commonly uses fixed-width numerical types.

Examples include:

```
1 np.int8
2 np.int16
3 np.int32
4 np.int64
```

and:

```
1 np.float32
2 np.float64
```

The number indicates the number of bits used to represent each value.

For example:

```
1 x = np.array([1, 2, 3], dtype=np.int32)
2
3 x.dtype
```

```
dtype('int32')
```

Each element occupies 32 bits:

$$32 \text{ bits} = 4 \text{ bytes.}$$

For large arrays, dtype therefore directly affects memory consumption.

13.1 Memory Consumption

Consider:

```
1 a = np.zeros(1_000_000, dtype=np.float64)
2 b = np.zeros(1_000_000, dtype=np.float32)
3
4 print(a.nbytes)
5 print(b.nbytes)
```

```
8000000
```

```
4000000
```

A float64 uses 8 bytes per element, while a float32 uses 4 bytes.

Approximately:

$$10^6 \times 8 = 8 \times 10^6 \text{ bytes,}$$

whereas:

$$10^6 \times 4 = 4 \times 10^6 \text{ bytes.}$$

Thus, reducing numerical precision can substantially reduce memory requirements.

However, lower precision also reduces the range and accuracy with which numbers can be represented.

13.2 Integer Range

Fixed-width integer types have bounded ranges.

For example:

```
1 np.iinfo(np.int8)
```

```
iinfo(min=-128, max=127, dtype=int8)
```

A signed 8-bit integer has range:

$$-128 \leq x \leq 127.$$

Values outside the representable range cannot be represented correctly using that dtype.

Larger integer types provide wider ranges at the cost of additional memory.

13.3 Floating-Point Precision

Floating-point arrays inherit the finite-precision behaviour discussed previously for Python floating-point numbers.

For example:

```
1 x = np.array([0.1, 0.2, 0.3])
```

```
2
```

```
3 x[0] + x[1] == x[2]
```

```
np.False_
```

may evaluate to False because these decimal values do not generally have exact binary floating-point representations.

Comparisons involving numerical calculations should therefore often use tolerances:

```
1 np.isclose(x[0] + x[1], x[2])
```

```
np.True_
```

For complete arrays:

```
1 a = np.array([0.1 + 0.2, 0.6])
2 b = np.array([0.3, 0.6])
3
4 np.allclose(a, b)
```

```
True
```

`np.isclose()` performs element-wise approximate comparisons, while `np.allclose()` determines whether all corresponding elements are sufficiently close.

These functions are preferable to exact equality when comparing floating-point results produced by numerical computation.

13.4 Type Conversion

An existing array can be converted to another dtype using `astype()`:

```
1 x = np.array([1.2, 2.7, 3.9])
2
3 x.astype(int)
```

```
array([1, 2, 3])
```

The conversion does not perform conventional rounding. The fractional component is discarded when converting these positive values to integers.

Converting in the opposite direction:

```
1 x = np.array([1, 2, 3])
2
3 x.astype(float)
```

```
array([1., 2., 3.])
```

produces floating-point representations.

Type conversion should therefore be considered part of the numerical meaning of a computation rather than merely a formatting operation.

14 Random Number Generation

Randomness is important in numerical computing and machine learning.

Typical applications include:

- initializing model parameters;
- sampling observations;
- generating synthetic data;
- shuffling datasets;
- creating train/test partitions;
- simulation;
- Monte Carlo methods.

Computer-generated random numbers are generally **pseudo-random**.

A pseudo-random number generator produces a deterministic sequence that behaves statistically like a random sequence.

The sequence is determined by an internal state.

14.1 Random Number Generators

Modern NumPy code can create an explicit random number generator using:

```
1 rng = np.random.default_rng()
```

Random values can then be generated through this object.

For example, random integers:

```
1 rng.integers(0, 10, size=5)
```

```
array([7, 1, 8, 8, 3])
```

generate five integers in the interval:

$[0, 10)$.

Random floating-point values from the interval $([0,1))$ can be generated with:

```
1 rng.random(5)

array([0.5649266 , 0.05572403, 0.29632918, 0.68975602, 0.08957462])
```

A multidimensional shape can be specified:

```
1 rng.random((2, 3))

array([[0.01210631, 0.78181098, 0.29127308],
       [0.37669424, 0.23805395, 0.94648192]])
```

14.2 Reproducibility and Seeds

Because pseudo-random number generation is deterministic, the generator can be initialized with a **seed**:

```
1 rng = np.random.default_rng(42)
2
3 rng.random(5)

array([0.77395605, 0.43887844, 0.85859792, 0.69736803, 0.09417735])
```

Creating another generator with the same seed:

```
1 rng = np.random.default_rng(42)
2
3 rng.random(5)

array([0.77395605, 0.43887844, 0.85859792, 0.69736803, 0.09417735])
```

produces the same sequence.

This property is essential for reproducibility.

A machine learning experiment may involve randomness in:

- data splitting;
- subsampling;
- initialization;
- stochastic optimization;
- simulation.

Controlling the seed allows the same pseudo-random decisions to be reproduced when debugging or comparing experiments.

However, a seed does not make an experiment conceptually non-random. It simply fixes one particular pseudo-random sequence.

14.3 Sampling from a Distribution

The generator can sample from common probability distributions.

For example, the standard normal distribution:

```

1 rng = np.random.default_rng(42)
2
3 samples = rng.normal(
4     loc=0,
5     scale=1,
6     size=1000
7 )
8
9 samples[:5]
```

```
array([ 0.30471708, -1.03998411,  0.7504512 ,  0.94056472, -1.95103519])
```

This corresponds to:

$$X \sim \mathcal{N}(0, 1).$$

A normal distribution with mean (μ) and standard deviation (σ) can be sampled using:

```

1 rng.normal(
2     loc=10,
3     scale=2,
4     size=1000
5 )
```

```

array([ 9.88143471, 8.54142613, 9.17105387, 11.26782076, 10.00598658,
       10.68041995, 11.34015848, 9.25031707, 11.51249633, 10.75768521,
       7.53037383, 12.88460958, 8.99851051, 6.68980931, 7.90991258,
       7.95790968, 10.10434664, 9.45229899, 9.32633606, 11.23940393,
       10.67975072, 10.63209574, 10.81965691, 11.23226934, 5.78409332,
       9.2711235 , 5.63957987, 10.07211985, 9.99073406, 12.09106455,
       12.37526879, 10.4055501 , 8.99927785, 10.97032223, 8.94416473,
       9.9972147 , 11.97227263, 8.88445731, 11.61135728, 11.35480274,
       8.09041637, 11.94778826, 11.39713269, 10.20385177, 8.47535308,
       8.28158796, 8.92467485, 11.08518893, 8.08874967, 10.8750245 ,
       7.51648894, 9.59186283, 10.21929551, 14.89026029, 7.24536869,
       12.94402913, 10.29968483, 10.82234384, 10.23669583, 10.88945364,
       9.69262979, 12.90810036, 9.08705771, 12.26446914, 8.71126576,
       9.87949631, 7.8560749 , 10.91004744, 12.89025191, 9.84528764,
       9.6062286 , 7.77076607, 9.54141469, 6.81441148, 8.17424403,
       10.45357209, 12.6380264 , 15.61842172, 8.82682983, 12.87060031,
       10.48750496, 9.69750471, 10.8651889 , 10.12383297, 10.22079145,
       9.18333367, 7.20378068, 6.91274959, 11.30648977, 9.44659982,
       8.8078222 , 10.01701125, 11.58986451, 10.36072789, 8.68789013,
       12.45258591, 13.15837101, 10.98911331, 11.94732702, 12.48391999,
       12.26019863, 11.22819835, 11.19669409, 11.03932115, 7.80532465,
       11.40144631, 7.28837206, 8.41080109, 12.60713014, 11.68044033,
       12.97479658, 9.45665123, 7.69550232, 9.51912561, 10.20406259,
       10.15797743, 12.26736096, 9.27745915, 10.70406894, 8.0227457 ,
       10.90063456, 10.0062171 , 8.50037332, 9.52834128, 9.63164851,
       9.45968974, 13.54268248, 9.803108 , 9.51220778, 5.80509808,
       8.2115853 , 9.47385315, 8.62829465, 12.76341519, 9.67012445,
       12.5768885 , 10.12340927, 10.07445567, 9.8224071 , 10.0075925 ,
       13.43759001, 5.36077855, 5.99690851, 8.91365834, 10.02905576,
       11.38020885, 10.9461744 , 9.23150817, 12.03808739, 12.06039081,
       10.36811566, 11.92531514, 10.54529053, 8.87703395, 11.39563368,
       10.22122625, 10.00267969, 12.94590714, 5.09825574, 7.16463166,
       7.62585915, 9.2734787 , 9.49078366, 6.9853487 , 8.02968394,
       8.27831664, 14.91484818, 13.6034839 , 9.17650116, 9.27283342,
       7.70159201, 6.18373917, 9.76623963, 8.00429603, 9.83029894,
       6.7995883 , 8.47605108, 10.29725419, 10.7324192 , 10.83494392,
       7.35902203, 11.70937146, 8.39957661, 11.26571583, 9.97872265,
       7.247225 , 9.36760459, 10.73072748, 11.22593076, 9.71802421,
       13.06355778, 12.01502154, 9.48669177, 11.50024362, 13.86758697,
       13.9209692 , 7.54400024, 8.14693378, 12.96983757, 7.8834379 ,
       7.35491773, 9.02761174, 10.84045342, 9.79520659, 8.69887276,
       8.65157508, 8.57532587, 8.24098074, 14.56326575, 10.59502214,
       11.77351811, 9.02184506, 9.62808665, 8.57289204, 4.69658333,
       7.24398145, 6.37876264, 5.50043244, 7.60928205, 12.64991136,
       9.91112766, 12.58112752, 10.82196515, 11.56511765, 8.19820601,
       11.04790191, 11.45742847, 8.84670542, 11.11908074, 11.13237552,
       8.90089476, 7.75446314, 7.63174334, 10.17301108, 10.65401974,
       8.41494834, 10.05317068, 11.14107407, 11.23638143, 13.10376024,
       12.55396166, 7.99741713, 13.36717242, 8.93804699, 12.08974562,
       10.13597325, 9.17570124, 6.38460628, 9.65687098, 6.88123353,

```

11.93459847, 13.03358531, 8.40617802, 10.60394149, 8.54921696,
 8.74303054, 11.54898941, 9.92292348, 13.47929608, 9.03553504,
 11.99867266, 10.19482683, 11.59155602, 9.10568469, 9.90088232,
 9.76900224, 8.32788577, 11.32416307, 11.30049801, 11.18710939,
 13.07393131, 12.8986896, 9.25011062, 10.74610464, 8.75279862,
 10.04671063, 8.8005011, 13.24014062, 10.7273963, 9.56394174,
 12.38723714, 5.65876918, 5.97179481, 11.59167731, 10.1209369,
 10.5008926, 7.34720779, 9.95353258, 14.00701882, 11.64839924,
 10.33941309, 9.12189218, 9.2108543, 5.74160422, 10.51528084,
 11.69096622, 6.84876135, 8.67716622, 7.68507074, 8.06929761,
 10.10696325, 5.83119569, 11.22854088, 11.50771014, 9.49779482,
 5.03858298, 8.00716275, 12.46580387, 4.44401144, 9.30495318,
 7.63999844, 11.6091406, 8.6497721, 10.80790804, 11.13092084,
 13.67245168, 9.59411611, 10.75208874, 7.02988904, 12.38082713,
 8.4789823, 8.87849167, 9.95550441, 6.87423131, 10.45754006,
 11.93493255, 10.58586765, 6.82220487, 9.75238141, 11.48405781,
 6.24402247, 7.85046797, 11.75308721, 10.54019087, 10.82394316,
 13.71898962, 10.93710556, 11.31577602, 15.42149105, 9.57239088,
 8.40423447, 10.8555928, 10.87798206, 8.94656922, 7.93520596,
 14.61968884, 10.65368694, 8.53542924, 11.8318606, 9.7817097,
 9.49423711, 11.12454574, 7.41623546, 10.70525441, 13.82590962,
 10.93235532, 9.24757579, 8.98578944, 9.17932005, 9.49207464,
 9.24438693, 11.83262288, 6.53358179, 11.7240001, 9.24062695,
 10.75389557, 7.74602418, 12.65726213, 12.49126992, 11.91918105,
 6.97049045, 11.65824214, 10.80971018, 6.78945751, 9.95140523,
 10.72906898, 11.11215055, 10.35452273, 10.58246283, 12.94722278,
 12.45206782, 4.26589009, 9.36512289, 9.67069794, 6.49580285,
 10.18836612, 12.49751554, 7.82686428, 10.67280188, 8.16835305,
 8.65617595, 12.95478727, 8.91488436, 10.92442498, 6.25756377,
 13.72463512, 11.20530489, 9.63500031, 11.15109302, 7.18036114,
 12.36506148, 9.35203045, 9.59186097, 9.01668877, 8.83869719,
 11.38934122, 10.5118137, 11.30349628, 9.98241062, 11.02639962,
 9.16230251, 14.45946368, 6.86413917, 12.52187465, 12.7358149,
 8.29568239, 14.45854722, 11.99098057, 11.94227801, 10.05604437,
 9.56554768, 9.58118235, 10.78106119, 12.80237255, 10.36262844,
 8.17711275, 11.64885412, 11.84230149, 11.30931906, 11.49099781,
 9.45814374, 8.14202167, 8.26361314, 6.7290735, 10.47389722,
 9.83310243, 9.12002459, 4.08876235, 7.50536506, 12.2416813,
 8.670748, 10.72118336, 7.21862914, 12.87741096, 9.71194468,
 10.58008079, 14.65155958, 13.0328802, 9.38686876, 8.81355963,
 9.34890998, 11.0197965, 9.78955173, 9.20951171, 12.9343625,
 9.77158022, 14.59917819, 10.18148513, 13.57063679, 10.71463885,
 11.92592115, 8.5507779, 8.29248155, 10.91941191, 11.97710648,
 10.080513, 7.42721333, 7.27948327, 7.1457272, 6.46475577,
 10.21897072, 12.25008784, 9.97316843, 5.47917049, 9.61847908,
 9.67440201, 14.1919975, 7.08572417, 9.21864479, 13.41170536,
 10.89995668, 11.12662615, 10.22916887, 10.65988708, 11.55521906,
 8.38102736, 11.75611716, 7.25399397, 10.65673595, 10.35748193,
 8.76741382, 9.95462945, 8.75491969, 7.85659735, 9.29277052,
 7.79202273, 10.64559493, 7.69982901, 11.42395405, 7.63704186,

8.86741017, 8.75079318, 12.65029346, 10.66057989, 9.57653239,
 10.99743888, 5.78561531, 9.91373282, 13.99502836, 10.26469301,
 10.13582634, 11.09625676, 11.79121049, 6.00506661, 6.63534704,
 10.42058258, 8.57024688, 12.62405052, 9.36965122, 6.13093853,
 7.46397217, 4.50064796, 9.53174893, 9.62323175, 9.15107458,
 12.15775808, 12.39703967, 9.65922315, 5.73061267, 11.23456316,
 11.11072046, 10.61878314, 11.9629595, 7.67034381, 14.01751077,
 8.08114449, 11.49844585, 6.62530461, 8.13947063, 13.03273729,
 9.47820035, 11.67984521, 10.47391078, 9.79008369, 12.72770065,
 9.52668235, 6.43852378, 8.39351982, 11.10238346, 10.79101821,
 6.8658835, 8.55131496, 8.8365573, 9.26906081, 7.24817282,
 8.74325778, 8.54277903, 11.54306652, 11.61824262, 9.70837908,
 9.21592041, 10.27984862, 8.36684979, 7.70633584, 8.11955758,
 8.73037367, 7.6730524, 10.32411912, 11.45943135, 6.8258034,
 10.05025742, 9.56012841, 10.68301737, 9.97412246, 7.09280144,
 7.4752214, 10.05884715, 12.05756184, 5.57162526, 10.69145316,
 12.37924401, 9.69394825, 11.10040499, 8.59386184, 9.7612676,
 6.75817296, 5.31131338, 11.8855316, 8.03001014, 9.08894431,
 7.22711549, 7.18006059, 9.6523246, 8.54084561, 10.755942,
 10.90507684, 10.64169929, 10.47720158, 13.15553641, 7.86481246,
 9.89013027, 12.26537964, 12.52907962, 7.1564727, 9.37528912,
 12.71624219, 11.01615821, 10.39052606, 9.56463061, 9.23239815,
 10.57643813, 9.49525175, 9.91484276, 10.15719741, 8.64385694,
 11.06425875, 15.8284893, 6.44602672, 10.64802505, 8.47961982,
 11.15480516, 10.09311495, 11.52370955, 12.35101315, 10.38484224,
 12.26410458, 5.31140421, 13.1809046, 10.7501243, 12.15568671,
 7.56024851, 8.33846024, 10.29187659, 9.96592422, 10.56613718,
 12.16903032, 6.46955937, 11.94763814, 10.01656382, 9.59243628,
 7.05776953, 11.00609734, 5.1717843, 11.29678802, 7.01578833,
 10.10691583, 9.76166233, 12.33907629, 9.64268966, 8.61172288,
 8.98679461, 12.54576554, 11.07259394, 9.22874004, 9.52710716,
 6.86564582, 10.7769484, 6.48036858, 11.50359293, 10.25724318,
 14.7739036, 7.79972889, 9.79467582, 13.5965754, 12.38277928,
 8.6075759, 10.44793486, 10.00399871, 10.56391173, 8.60263913,
 9.82135919, 6.80389525, 10.2702469, 10.90004135, 9.54516117,
 6.76989822, 11.97706373, 10.72905008, 8.5479698, 9.43879911,
 6.99866119, 7.94917974, 8.92437731, 12.45684162, 8.72099322,
 11.28042732, 8.92336759, 10.49589512, 9.717209, 11.7461132,
 8.93037732, 6.8549435, 7.23610194, 8.93624501, 7.50188006,
 10.82042403, 9.06846367, 14.7492131, 6.52457081, 10.40726636,
 7.74409395, 9.01319732, 10.28283895, 5.34925725, 10.47365223,
 13.80581492, 9.0931234, 10.51961571, 12.04984339, 9.672093,
 8.83807962, 5.44852719, 11.28198144, 9.77366318, 7.9746323,
 10.25319342, 8.83263929, 8.94314774, 11.21258231, 7.32323518,
 12.09463057, 7.20462767, 8.67459718, 13.84589239, 12.25470952,
 10.19520798, 8.11893957, 8.94145539, 10.14655063, 10.07504646,
 9.81825814, 9.93833497, 9.57124978, 7.11604253, 8.96697852,
 11.82924811, 7.2633314, 9.62088898, 8.36049214, 10.22270086,
 9.22273364, 12.95478733, 11.01316631, 7.37639367, 11.34245602,
 8.45069629, 8.84600726, 10.62763804, 9.01650516, 11.78571163,

8.45097988, 8.76210689, 8.68725821, 8.8687225, 12.19838311,
 11.25759259, 13.41988791, 14.53528841, 7.93856594, 10.14083825,
 7.64759937, 8.44478814, 8.73983498, 7.88569025, 7.89622545,
 9.71143124, 7.56656316, 11.68425392, 8.16449293, 15.61913794,
 9.0972311, 10.14403528, 7.40931763, 13.41218352, 12.23900136,
 10.59383174, 7.3063949, 10.42133376, 7.75713791, 12.92471053,
 11.61838946, 10.19087808, 5.12724867, 8.03316804, 11.37552226,
 12.42576738, 12.21833962, 6.815974, 10.60310461, 12.03103043,
 6.68332887, 9.98419097, 10.5476848, 9.39847083, 6.29567629,
 9.34860378, 9.34538914, 9.70385519, 8.87305715, 9.11397569,
 9.92058744, 10.88490921, 9.39114647, 9.18070214, 10.18470808,
 6.15816815, 10.95407025, 9.99887381, 10.07938705, 11.92146862,
 6.09436894, 6.05233226, 10.54760769, 9.98576629, 10.34035389,
 9.07574585, 11.32870735, 11.86527769, 10.96796459, 9.47834308,
 5.78462122, 11.53847242, 12.44345733, 11.28911682, 8.24071515,
 11.34758879, 11.72105942, 13.1176325, 8.24778015, 9.54709083,
 11.4945441, 10.11482014, 7.7653448, 10.82923257, 11.15732327,
 10.17592579, 10.43916985, 9.4825882, 12.95655787, 8.19861563,
 8.82535847, 10.02197103, 11.75790806, 7.00525366, 7.3794065,
 5.92454298, 10.06539045, 7.76338837, 9.23089978, 11.32275638,
 8.71691456, 10.46256405, 9.40132418, 11.34422863, 10.95025613,
 8.61613261, 9.66269649, 12.0874868, 10.78675374, 7.20607588,
 11.31123612, 7.83077876, 12.82502614, 12.04819768, 10.57977095,
 10.31446569, 9.58193549, 11.3570479, 9.25844366, 10.47113974,
 11.74386873, 8.87638691, 8.76761452, 7.61126904, 7.87105451,
 6.34320938, 11.45965758, 13.61782908, 12.85065789, 6.74783102,
 11.44970934, 12.8442127, 9.08638004, 10.20289486, 5.9842139,
 10.53347234, 7.87288054, 10.39927004, 6.87759416, 11.08571645,
 12.04902535, 8.43364028, 7.72615484, 5.68795391, 9.46717293,
 8.83681213, 6.89796118, 10.90603751, 9.10005924, 7.84852808,
 8.89085519, 12.14260908, 11.20587389, 9.73994788, 11.23821004,
 7.67817609, 10.0647402, 10.29464466, 9.06032698, 10.87091667,
 8.43913309, 11.01964142, 11.10371754, 4.16568024, 12.38061141,
 3.90473487, 8.76790195, 12.05069607, 9.36542655, 7.06762804,
 7.81252588, 12.05381149, 6.15699094, 12.2579822, 9.8897958,
 7.61520934, 8.83376986, 7.96555132, 11.7324504, 11.46485562,
 6.56167773, 9.19281576, 8.27362868, 12.58701994, 7.29980176,
 13.20845821, 9.93341052, 7.67613119, 8.64545163, 8.39676232,
 7.4507632, 10.18965372, 7.5441689, 5.63926758, 9.24064706,
 6.39428761, 8.49567826, 8.18179422, 8.25756786, 14.0019262,
 10.11282313, 10.49074202, 6.44720341, 8.69207373, 10.43060526,
 7.95094217, 11.81612569, 7.59460683, 9.42726773, 8.82789672,
 10.32864132, 8.71901986, 9.45821275, 9.52773982, 8.53122449,
 10.97018774, 10.47677675, 9.82277211, 9.13908333, 11.59308806,
 9.52833151, 9.38660542, 10.20938006, 13.39317337, 7.31928758,
 11.53971379, 6.21365768, 10.43454632, 13.15016566, 10.97733705,
 10.34125016, 8.87910106, 10.0620944, 9.49587437, 8.42804678,
 9.42024877, 10.39676232, 8.46556586, 8.38743636, 7.27951312,
 10.78145005, 11.09064652, 10.46332089, 11.93034099, 10.17573527,
 10.78402187, 10.60767485, 9.81691996, 8.93835634, 14.42830523])

corresponding to:

$$X \sim \mathcal{N}(10, 2^2).$$

Uniform samples can be generated with:

```
1 rng.uniform(  
2     low=-1,  
3     high=1,  
4     size=5  
5 )
```

```
array([-0.91530777, -0.11012448,  0.09397558, -0.70651353, -0.32742834])
```

Random sampling therefore provides a direct computational connection between probability distributions and numerical arrays.

14.4 Random Choice

Elements can also be sampled from an existing array:

```
1 values = np.array([10, 20, 30, 40, 50])  
2  
3 rng.choice(values)
```

```
np.int64(10)
```

Several values can be sampled:

```
1 rng.choice(values, size=3)
```

```
array([50, 10, 10])
```

Sampling probabilities can be specified explicitly:

```

1 values = np.array([0, 1])
2
3 rng.choice(
4     values,
5     size=10,
6     p=[0.7, 0.3]
7 )

```

```
array([0, 1, 0, 0, 1, 0, 0, 0, 1, 0])
```

Here:

$$P(X = 0) = 0.7, \quad P(X = 1) = 0.3.$$

The probability vector must satisfy:

$$p_i \geq 0$$

and:

$$\sum_i p_i = 1.$$

14.5 Sampling With and Without Replacement

By default, `choice()` samples with replacement.

This means that the same element can appear multiple times:

```

1 rng.choice(
2     values,
3     size=5,
4     replace=True
5 )

```

```
array([1, 1, 0, 0, 1])
```

Sampling without replacement can be requested explicitly:

```

1 values = np.array([10, 20, 30, 40, 50])
2
3 rng.choice(
4     values,
5     size=3,
6     replace=False
7 )

```

```
array([20, 10, 50])
```

Without replacement, each selected element is removed from the available sampling pool.

This distinction is important in statistical sampling and resampling procedures.

14.6 Permutations and Shuffling

A permutation rearranges the order of elements.

NumPy can generate a permuted copy:

```

1 values = np.array([1, 2, 3, 4, 5])
2
3 rng.permutation(values)

```

```
array([2, 4, 1, 3, 5])
```

The original array remains unchanged.

By contrast, shuffling modifies the array in place:

```

1 values = np.array([1, 2, 3, 4, 5])
2
3 rng.shuffle(values)
4
5 values

```

```
array([3, 2, 5, 4, 1])
```

The distinction is similar to the copy-versus-mutation concepts considered earlier.

In machine learning, shuffling observations before training is common when the original data order contains structure that should not determine the learning sequence.

15 Arrays in Data Science and Machine Learning

NumPy arrays form one of the conceptual foundations of the scientific Python ecosystem.

Even when higher-level libraries provide specialized data structures, much of the underlying numerical reasoning remains array-based.

A typical supervised learning dataset can be represented as:

$$X \in \mathbb{R}^{n \times p},$$

where:

- (n) is the number of observations;
- (p) is the number of features.

The corresponding target vector may be represented as:

$$\mathbf{y} \in \mathbb{R}^n.$$

For example:

```
1 X = np.array([
2     [1.2, 3.4, 5.6],
3     [2.1, 3.8, 5.2],
4     [0.8, 2.9, 4.7],
5     [1.7, 3.2, 5.0]
6 ])
7
8 y = np.array([0, 1, 0, 1])
9
10 print(X.shape)
11 print(y.shape)
```

```
(4, 3)
(4,)
```

The structural relationship:

```
1 X.shape == (4, 3)
2 y.shape == (4,)
```

expresses the fact that four observations have three features each and one target value each.

15.1 Feature Transformation

Suppose each feature must be standardized:

$$z_{ij} = \frac{x_{ij} - \mu_j}{s_j}. \quad (12)$$

The feature means can be computed as:

```
1 feature_means = X.mean(axis=0)
```

and the feature standard deviations as:

```
1 feature_stds = X.std(axis=0)
```

Then:

```
1 X_standardized = (  
2     X - feature_means  
3 ) / feature_stds
```

combines:

- aggregation along axis=0;
- broadcasting;
- vectorized subtraction;
- vectorized division.

No explicit loops over observations or features are required.

This compact expression represents a complete numerical transformation:

$$Z = (X - \mu) \oslash \mathbf{s},$$

where the subtraction and division are applied feature-wise through broadcasting.

15.2 Linear Models

Array operations also map naturally to linear algebra.

For a linear model:

$$\hat{\mathbf{y}} = X\beta, \quad (13)$$

where:

$$X \in \mathbb{R}^{n \times p}$$

and:

$$\beta \in \mathbb{R}^p,$$

the prediction can be expressed directly as:

```
1 beta = np.array([0.5, -0.2, 0.8])
2
3 predictions = X @ beta
4
5 predictions
```

```
array([4.4 , 4.45, 3.58, 4.21])
```

The shapes are:

```
1 X:      (n, p)
2 beta:   (p,)
3 predictions: (n,)
```

Understanding the shape relationship allows us to reason about whether the operation is mathematically valid before executing it.

15.3 Model Evaluation

Suppose:

```

1 predictions = np.array([2.8, 3.5, 4.1, 5.0])
2 targets = np.array([3.0, 3.2, 4.4, 4.8])

```

The residual vector is:

```

1 residuals = targets - predictions
2
3 residuals

```

```
array([ 0.2, -0.3,  0.3, -0.2])
```

The mean squared error can then be written as:

```

1 mse = np.mean(residuals ** 2)
2
3 mse

```

```
np.float64(0.065000000000000013)
```

Mathematically:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2. \quad (14)$$

Again, the implementation closely resembles the mathematical definition.

This correspondence between mathematical expressions and array operations is one of the principal reasons NumPy is so useful in numerical and statistical programming.

15.4 Shape as Part of the Interface

In ordinary Python functions, we often reason primarily about types.

In numerical computing, the **shape** of an array is frequently just as important as its dtype.

For example, all of the following arrays contain three numbers:

```

1 a = np.array([1, 2, 3])
2
3 b = np.array([[1, 2, 3]])
4
5 c = np.array([
6     [1],
7     [2],
8     [3]
9 ])
10
11 print(a.shape)
12 print(b.shape)
13 print(c.shape)

```

```

(3,)
(1, 3)
(3, 1)

```

Their shapes are:

```

1 (3,)
2 (1, 3)
3 (3, 1)

```

They are therefore not interchangeable in every operation.

The same numerical values can represent:

- a one-dimensional vector;
- one row containing three values;
- three rows containing one value each.

This distinction is particularly important when debugging machine learning pipelines.

Many array-related errors are fundamentally **shape errors** rather than arithmetic errors.

For this reason, when inspecting unfamiliar numerical code, it is useful to ask:

1. What does each axis represent?
2. What is the shape of each array?
3. Which axis is being indexed or reduced?
4. Is broadcasting occurring?
5. Is an operation element-wise or algebraic?

6. Are any arrays sharing memory?
7. What dtype and numerical precision are being used?

These questions provide a systematic framework for reasoning about array computations.

16 Key Takeaways

NumPy's ndarray provides a homogeneous, multidimensional representation designed for numerical computation.

An array is characterized not only by its values but also by structural properties including:

- dimensionality;
- shape;
- size;
- data type.

Indexing and slicing allow individual elements and structured subsets to be selected. Integer indexing can reduce dimensionality, while slicing can preserve dimensions.

Boolean indexing combines vectorized comparisons with selection and provides a natural mechanism for filtering data.

Reshaping changes the structural interpretation of an array without changing its number of elements. Understanding shapes is central to numerical reasoning, particularly in machine learning.

NumPy distinguishes between assignments, views, and copies. Views can share underlying memory with other arrays, allowing efficient computation but also introducing shared mutation.

Vectorization expresses numerical operations over complete arrays rather than through explicit Python loops. It often produces code that is both more efficient and closer to mathematical notation.

Broadcasting allows operations between arrays with compatible but different shapes. Its rules are based on comparing dimensions from right to left and determining whether dimensions are equal or one of them is 1.

Universal functions apply scalar-style numerical operations efficiently over arrays. They provide many of NumPy's element-wise mathematical, arithmetic, and comparison operations.

Aggregation functions reduce arrays along specified axes. The axis argument identifies the dimension being collapsed, making axis semantics particularly important in multidimensional data.

The dtype determines numerical representation, memory consumption, range, and precision. Fixed-width numerical representations introduce considerations such as integer range and floating-point approximation.

Pseudo-random number generators provide reproducible randomness when initialized with a known seed. Random sampling, shuffling, and probability distributions are important tools in statistical and machine learning workflows.

Ultimately, effective use of arrays depends less on memorizing NumPy methods than on developing the ability to reason about **shape, axes, broadcasting, memory, dtype, and the mathematical meaning of an operation.**

17 Problems

The following problems review the main concepts introduced in this chapter. They progress from basic array construction and indexing to shape reasoning, broadcasting, numerical behaviour, and applications in data science and machine learning.

17.1 Problem 1 — Creating and Inspecting Arrays

Create the following NumPy array:

```
1 values = [2, 4, 6, 8, 10]
```

Convert it into an ndarray and inspect:

- its type;
- number of dimensions;
- shape;
- size;
- dtype.

Explain the difference between ndim, shape, and size.

17.2 Problem 2 — One-Dimensional and Two-Dimensional Arrays

Create the following arrays:

```

1 a = np.array([1, 2, 3, 4, 5, 6])
2
3 A = np.array([
4     [1, 2, 3],
5     [4, 5, 6]
6 ])

```

For each array, determine:

- ndim;
- shape;
- size.

Then explain why both arrays contain six elements but represent different structures.

17.3 Problem 3 — Array Creation Functions

Create:

1. a one-dimensional array containing five zeros;
2. a 3×4 array containing ones;
3. a 2×3 array filled with the value 7;
4. an array containing the integers from 0 through 9;
5. an array containing the even integers from 2 through 20;
6. an array containing exactly six evenly spaced values between 0 and 1;
7. a 4×4 identity matrix.

For each case, choose an appropriate NumPy array-creation function.

Explain when `np.linspace()` may be preferable to `np.arange()` for floating-point intervals.

17.4 Problem 4 — Indexing

Consider:

```

1 A = np.array([
2     [10, 20, 30, 40],
3     [50, 60, 70, 80],
4     [90, 100, 110, 120]
5 ])

```


Without executing the code, determine the values returned by:

```
1 A[0, 0]
2 A[1, 2]
3 A[-1, -1]
4 A[2, 1]
5 A[0]
6 A[:, 2]
```

For each expression, determine whether the result is:

- a scalar;
- a one-dimensional array;
- a two-dimensional array.

17.5 Problem 5 — Slicing

Using the same array:

```
1 A = np.array([
2     [10, 20, 30, 40],
3     [50, 60, 70, 80],
4     [90, 100, 110, 120]
5 ])
```

write expressions that select:

1. the first two rows;
2. the last two columns;
3. the first two rows and the middle two columns;
4. the second row only;
5. the second row while preserving two dimensions;
6. every second column.

Explain the difference between:

```
1 A[1]
```

and:

```
1 A[1:2]
```

in terms of shape.

17.6 Problem 6 — Shape Preservation

Consider:

```
1 X = np.array([
2     [1.2, 2.3, 3.4],
3     [4.5, 5.6, 6.7],
4     [7.8, 8.9, 9.0]
5 ])
```

Determine the shape of each expression:

```
1 X[0]
2 X[0:1]
3 X[:, 0]
4 X[:, 0:1]
5 X[1:, :2]
```

Explain why preserving or removing a dimension can matter when passing arrays to machine learning code.

17.7 Problem 7 — Boolean Masks

Given:

```
1 values = np.array([3, 8, 12, 5, 17, 21, 4, 15])
```

construct boolean masks that identify:

1. values greater than 10;
2. values between 5 and 15, inclusive;
3. even values;
4. values smaller than 5 or greater than 15.

Use each mask to extract the corresponding values.

17.8 Problem 8 — Filtering Observations

Suppose:

```
1 X = np.array([
2     [22, 42000],
3     [35, 58000],
4     [48, 76000],
5     [29, 51000],
6     [52, 91000]
7 ])
```

The first column represents age and the second represents annual income.

Select all observations satisfying:

$$\text{age} \geq 30$$

and:

$$\text{income} < 80000.$$

Your result should contain complete rows, not only the columns used in the condition.

Explain how a one-dimensional boolean mask can be used to filter a two-dimensional array by observations.

17.9 Problem 9 — Reshaping

Create:

```
1 a = np.arange(24)
```

Reshape it into arrays with the following shapes:

```
1 (6, 4)
2 (4, 6)
3 (2, 3, 4)
4 (3, 2, 4)
```

Then determine which of the following reshapes are valid:

```
1 (8, 3)
2 (5, 5)
3 (2, 2, 6)
4 (3, 3, 3)
```

Justify each answer using the total number of elements.

17.10 Problem 10 — Inferring Dimensions

Given:

```
1 a = np.arange(30)
```

predict the shape produced by:

```
1 a.reshape(5, -1)
2 a.reshape(-1, 3)
3 a.reshape(2, -1, 5)
```

Explain how NumPy determines the dimension represented by -1.

Why can only one dimension normally be specified as -1?

17.11 Problem 11 — Flattening

Consider:

```
1 A = np.array([
2     [1, 2, 3],
3     [4, 5, 6]
4 ])
```

Use each of the following approaches to produce a one-dimensional array:

```
1 A.reshape(-1)
2 A.ravel()
3 A.flatten()
```

Determine the shape of each result.

Then explain the important memory-related distinction between `ravel()` and `flatten()`.

17.12 Problem 12 — Assignment, View, or Copy?

Consider:

```
1 a = np.array([10, 20, 30, 40, 50])
2
3 b = a
4 c = a[1:4]
5 d = a[1:4].copy()
```

Without executing the code, predict what happens to a, b, c, and d after:

```
1 b[0] = 100
2 c[0] = 200
3 d[0] = 300
```

Explain the distinction between:

- another name for the same array;
- a view;
- an independent copy.

17.13 Problem 13 — Why Views Matter

Suppose a very large array contains several gigabytes of numerical data.

Explain why NumPy might prefer to return views from slicing operations rather than automatically copying the selected values.

Discuss the trade-off between:

- memory efficiency;
- computational efficiency;
- independence of modifications.

17.14 Problem 14 — Element-Wise Operations

Given:

```
1 a = np.array([1, 2, 3, 4])
2 b = np.array([10, 20, 30, 40])
```

determine the results of:

```
1 a + b
2 a - b
3 a * b
4 b / a
5 a ** 2
```

Explain why these operations are described as element-wise.

17.15 Problem 15 — Element-Wise Versus Matrix Multiplication

Consider:

```
1 A = np.array([
2     [1, 2],
3     [3, 4]
4 ])
5
6 B = np.array([
7     [5, 6],
8     [7, 8]
9 ])
```

Calculate:

```
1 A * B
```

and:

```
1 A @ B
```

by hand.

Explain the mathematical difference between the two operations.

17.16 Problem 16 — Vectorization

Suppose:

```
1 x = np.array([1, 2, 3, 4, 5])
```

The desired transformation is:

$$y_i = 3x_i^2 - 2x_i + 1.$$

Implement the transformation:

1. using an explicit Python loop;
2. using a vectorized NumPy expression.

Compare the two implementations in terms of:

- readability;
- mathematical correspondence;
- computational abstraction.

17.17 Problem 17 — Broadcasting with a Scalar

Given:

```
1 x = np.array([2, 4, 6, 8])
```

predict the results of:

```
1 x + 10
2 x * 2
3 x / 4
```

Explain broadcasting conceptually without describing NumPy as physically constructing a full array of repeated scalar values.

17.18 Problem 18 — Broadcasting Across Rows

Consider:

```

1 X = np.array([
2     [1, 2, 3],
3     [4, 5, 6],
4     [7, 8, 9],
5     [10, 11, 12]
6 ])
7
8 offset = np.array([10, 20, 30])

```

Predict the result of:

```

1 X + offset

```

Then explain why the shapes:

```

1 (4, 3)
2 (3,)

```

are compatible for broadcasting.

17.19 Problem 19 — Broadcasting Rules

Determine whether each pair of shapes is broadcast-compatible.

If they are compatible, determine the resulting shape.

```

1 (4, 3)    and (3,)
2 (4, 3)    and (1, 3)
3 (4, 3)    and (4, 1)
4 (5, 4, 3) and (4, 1)
5 (2, 3, 4) and (3, 4)
6 (2, 3, 4) and (2, 4)
7 (6, 1, 5) and (1, 4, 1)

```

For each case, compare dimensions from right to left and justify the result.

17.20 Problem 20 — Row and Column Broadcasting

Create:

```
1 row = np.array([1, 2, 3])
2
3 column = np.array([
4     [10],
5     [20],
6     [30],
7     [40]
8 ])
```

Determine:

```
1 row.shape
2 column.shape
```

Then predict the shape and values of:

```
1 column + row
```

Explain why the result is a two-dimensional array even though neither input has that final shape.

17.21 Problem 21 — Universal Functions

Given:

```
1 x = np.array([1.0, 4.0, 9.0, 16.0])
```

calculate:

```
1 np.sqrt(x)
2 np.log(x)
3 np.exp(np.array([0.0, 1.0, 2.0]))
```

Then implement the sigmoid function:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

as a vectorized NumPy expression and evaluate it on:

```
1 x = np.array([-2.0, -1.0, 0.0, 1.0, 2.0])
```

17.22 Problem 22 — Aggregations

Given:

```
1 X = np.array([
2     [1, 2, 3],
3     [4, 5, 6],
4     [7, 8, 9],
5     [10, 11, 12]
6 ])
```

calculate:

```
1 X.sum()
2 X.mean()
3 X.min()
4 X.max()
```

Then determine the result and shape of:

```
1 X.sum(axis=0)
2 X.sum(axis=1)
3 X.mean(axis=0)
4 X.mean(axis=1)
```

Explain what is being reduced in each case.

17.23 Problem 23 — Reasoning About axis

Suppose:

```
1 X.shape == (1000, 20)
```

where X represents 1000 observations and 20 features.

Without executing code, determine the shape and interpretation of:

```
1 X.mean(axis=0)
2 X.std(axis=0)
3 X.mean(axis=1)
4 X.max(axis=1)
```

Avoid explaining axis=0 simply as “columns” or axis=1 simply as “rows.”

Instead, describe each operation in terms of **which dimension is being collapsed**.

17.24 Problem 24 — keepdims

Using:

```
1 X = np.array([
2     [1, 2, 3],
3     [4, 5, 6],
4     [7, 8, 9]
5 ])
```

compare the shapes produced by:

```
1 X.mean(axis=0)
2 X.mean(axis=0, keepdims=True)
3
4 X.mean(axis=1)
5 X.mean(axis=1, keepdims=True)
```

Explain why retaining a dimension can simplify subsequent broadcasting operations.

17.25 Problem 25 — Population and Sample Standard Deviation

Given:

```
1 x = np.array([4, 7, 8, 10, 11])
```

calculate:

```
1 x.std()
```

and:

```
1 x.std(ddof=1)
```

Explain why the results differ.

Identify which corresponds to:

$$\sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2}$$

and which corresponds to:

$$\sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}.$$

17.26 Problem 26 — Data Types and Memory

Create two arrays containing one million zeros:

```
1 a
2 b
```

where:

- a uses float64;
- b uses float32.

Use nbytes to determine the memory consumed by each.

Explain why the difference occurs and discuss the trade-off involved in using lower-precision floating-point types.

17.27 Problem 27 — Integer Representation

Investigate:

```
1 np.iinfo(np.int8)
```

and determine the minimum and maximum representable values.

Then explain why fixed-width NumPy integers differ conceptually from Python's built-in arbitrary-precision integers.

What kind of numerical problem can occur when a calculation exceeds the representable range of a fixed-width integer type?

17.28 Problem 28 — Floating-Point Comparison

Consider:

```
1 a = np.array([0.1 + 0.2, 0.6])
2 b = np.array([0.3, 0.6])
```

Predict the result of:

```
1 a == b
```

Then compare it with:

```
1 np.isclose(a, b)
```

and:

```
1 np.allclose(a, b)
```

Explain why exact equality is often inappropriate for comparing floating-point numerical results.

17.29 Problem 29 — Type Conversion

Given:

```
1 x = np.array([1.2, 2.7, 3.9, -4.8])
```

predict the result of:

```
1 x.astype(int)
```

Does NumPy round each value to the nearest integer?

Explain what information is lost during this conversion.

17.30 Problem 30 — Reproducible Random Numbers

Create two separate random number generators:

```
1 rng1 = np.random.default_rng(42)
2 rng2 = np.random.default_rng(42)
```

Generate five random values from each.

What relationship do you expect between the two sequences?

Then create another generator using a different seed and compare the result.

Explain the role of a seed in reproducible numerical experiments.

17.31 Problem 31 — Sampling from a Normal Distribution

Generate 10,000 observations from:

$$X \sim \mathcal{N}(5, 2^2).$$

Calculate the sample mean and sample standard deviation.

Compare them with the theoretical parameters:

$$\mu = 5, \quad \sigma = 2.$$

Should the sample statistics be exactly equal to the theoretical values?

Explain why or why not.

17.32 Problem 32 — Sampling with and without Replacement

Given:

```
1 values = np.array([10, 20, 30, 40, 50])
```

sample:

1. five values with replacement;
2. five values without replacement.

Can repeated values occur in each case?

Then determine whether the following request is valid:

```
1 rng.choice(  
2     values,  
3     size=10,  
4     replace=False  
5 )
```

Explain your answer.

17.33 Problem 33 — Feature Centering

Suppose:

```
1 X = np.array([  
2     [10.0, 100.0, 1000.0],  
3     [12.0, 120.0, 1100.0],  
4     [14.0, 140.0, 1200.0],  
5     [16.0, 160.0, 1300.0]  
6 ])
```

Calculate the mean of each feature and create a centered feature matrix:

$$X_{\text{centered}} = X - \mu.$$

Use vectorized NumPy operations and broadcasting.

Then verify that:

```
1 X_centered.mean(axis=0)
```

is numerically close to zero.

Explain the shapes involved in the operation.

17.34 Problem 34 — Feature Standardization

Using the same feature matrix, calculate:

$$z_{ij} = \frac{x_{ij} - \mu_j}{s_j}.$$

Create a standardized matrix using:

```
1 feature_means = ...  
2 feature_stds = ...  
3 X_standardized = ...
```

Do not use explicit Python loops.

Verify that each transformed feature has approximately:

$$\text{mean} = 0$$

and:

$$\text{standard deviation} = 1.$$

Explain how aggregation, broadcasting, and vectorization interact in this single computation.

17.35 Problem 35 — Linear Prediction with Arrays

Suppose:


```

1 X = np.array([
2     [1.0, 2.0, 3.0],
3     [2.0, 1.0, 4.0],
4     [3.0, 0.0, 2.0],
5     [4.0, 1.0, 1.0]
6 ])
7
8 beta = np.array([0.5, -0.3, 0.8])

```

Calculate:

```

1 X @ beta

```

Before executing the operation, reason about the shapes:

```

1 X.shape
2 beta.shape

```

and determine the expected shape of the result.

Then interpret the operation as:

$$\hat{\mathbf{y}} = X\beta.$$

17.36 Problem 36 — Residuals and Mean Squared Error

Given:

```

1 targets = np.array([3.0, 4.5, 2.8, 5.2])
2 predictions = np.array([2.7, 4.8, 3.1, 4.9])

```

calculate the residuals:

$$e_i = y_i - \hat{y}_i,$$

and then calculate:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n e_i^2.$$

Implement the calculation using vectorized NumPy operations.

Do not use an explicit loop.

17.37 Problem 37 — Debugging a Shape Error

Suppose:

```
1 X = np.zeros((100, 5))
2 beta = np.zeros((4,))
```

A program attempts:

```
1 predictions = X @ beta
```

Explain why the operation fails.

Identify the incompatible dimensions and determine what shape beta would need for the operation to produce one prediction per observation.

17.38 Problem 38 — Broadcasting Bug

Consider:

```
1 X = np.zeros((100, 5))
2 offset = np.zeros((100,))
```

A programmer expects:

```
1 X + offset
```

to add one offset value to every observation.

Will this operation broadcast successfully?

Explain your answer using the broadcasting rules.

Then propose an appropriate shape for offset if the intention is to add one different scalar value to each row.

17.39 Problem 39 — Diagnosing Unexpected Mutation

Consider:

```
1 X = np.arange(12).reshape(3, 4)
2
3 subset = X[:, :2]
4
5 subset[:, :] = 0
```

Without executing the code, predict what happens to `X`.

Explain why.

Then modify the code so that changing `subset` does not alter the original array.

17.40 Problem 40 — Integrated Array Reasoning

Consider:

```
1 X = np.array([
2     [2.0, 10.0, 100.0],
3     [4.0, 20.0, 120.0],
4     [6.0, 30.0, 140.0],
5     [8.0, 40.0, 160.0]
6 ])
7
8 means = X.mean(axis=0)
9 stds = X.std(axis=0)
10
11 Z = (X - means) / stds
12
13 mask = Z[:, 0] > 0
14
15 result = Z[mask]
```

Without executing the code:

1. determine the shape of `X`;
2. determine the shape of `means`;
3. determine the shape of `stds`;

4. explain how X - means is broadcast;
5. determine the shape of Z;
6. determine the shape of mask;
7. determine how many rows are likely selected by mask;
8. determine the shape of result.

Then describe, in conceptual terms, what the complete computation is doing.

Your explanation should explicitly use the concepts of:

- aggregation;
- axis reduction;
- broadcasting;
- vectorization;
- boolean indexing;
- shape.

17.41 Problem 41 — Designing an Array-Based Preprocessing Function

Write a function with the interface:

```

1 def standardize_features(
2     X: np.ndarray
3 ) -> np.ndarray:
4     ...

```

The function should:

- calculate one mean per feature;
- calculate one standard deviation per feature;
- return a standardized copy of the data;
- avoid modifying the original array;
- avoid explicit Python loops;
- raise an appropriate error if any feature has zero standard deviation.

Add a concise docstring.

Then explain why the function can be understood primarily through the shapes:

```

1 X          (n_samples, n_features)
2 feature_means (n_features,)
3 feature_stds (n_features,)
4 result      (n_samples, n_features)

```

17.42 Problem 42 — Reasoning About an Unfamiliar Numerical Pipeline

Consider:

```
1 def transform(X):
2     selected = X[X[:, 0] > 0]
3
4     means = selected.mean(
5         axis=0,
6         keepdims=True
7     )
8
9     centered = selected - means
10
11     scale = np.sqrt(
12         np.mean(centered ** 2, axis=0)
13     )
14
15     return centered / scale
```

Without executing the function, explain each stage.

In particular:

1. What does $X[:, 0] > 0$ produce?
2. What does $X[X[:, 0] > 0]$ select?
3. Why does `means` have shape $(1, n_features)$?
4. How is `selected - means` broadcast?
5. What does `centered ** 2` do?
6. Which dimension is reduced by `axis=0`?
7. What is the shape of `scale`?
8. How does the final division broadcast?
9. Does the function modify the original `X`?
10. What potential numerical problem occurs if one feature has zero scale?

Finally, summarize the transformation in mathematical terms.